



**Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Пензенский государственный университет»
(ФГБОУ ВО «Пензенский государственный университет»)
Нижнеломовский филиал ФГБОУ ВО
«Пензенский государственный университет»
(НлФ ФГБОУ ВО «ПГУ»)**

**МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ
ДЛЯ СТУДЕНТОВ ПО ВЫПОЛНЕНИЮ
ЛАБОРАТОРНЫХ РАБОТ**

**МДК 03.02. ИНСТРУМЕНТАЛЬНЫЕ СРЕДСТВА РАЗРАБОТКИ ПО
ПМ.03. УЧАСТИЕ В ИНТЕГРАЦИИ ПРОГРАММНЫХ МОДУЛЕЙ**

Специальность 09.02.07 «Информационные системы и программирование»

Нижний Ломов

2025

Содержание

Название лабораторных работ	страницы
Лабораторная работа №1 «Анализ предметной области ИС»	4
Лабораторная работа №2 «Применение структурного подхода в анализе требований и определении спецификаций ПО. Диаграмма переходов состояний».	8
Лабораторная работа №3 «Применение структурного подхода в анализе требований и определении спецификаций ПО. Функциональная диаграмма SADT».	13
Лабораторная работа №4 «Применение структурного подхода в анализе требований и определении спецификаций ПО. Диаграмма потоков данных».	20
Лабораторная работа №5 «Применение структурного подхода в анализе требований и определении спецификаций ПО. Диаграмма «сущность-связь».	28
Лабораторная работа №6 «Проектирование ПО при структурном подходе. Структурная схема».	33
Лабораторная работа №7 «Проектирование ПО при структурном подходе. Функциональная схема».	36
Лабораторная работа №8 «Применение объектно-ориентированного подхода в анализе и проектировании ПО. Диаграмма вариантов использования».	40
Лабораторная работа №9 «Применение объектно-ориентированного подхода в анализе и проектировании ПО. Диаграмма деятельности».	48
Лабораторная работа №10 «Применение объектно-ориентированного подхода в анализе и проектировании ПО. Диаграмма последовательности».	54
Лабораторная работа №11 «Применение объектно-ориентированного подхода в анализе и проектировании ПО. Диаграмма классов».	58

Структурный подход к проектированию ПО

На основании технического задания из практической работы дисциплины «Документирование и сертификация» выполнить структурный анализ функциональных требований к программному обеспечению и разработать необходимые диаграммы. Оформить результаты, используя MS Visio.

А также на основании технического задания и анализа требований к программному обеспечению в предыдущих работах разработать структурные и функциональные схемы программного обеспечения по своему варианту задания.

Лабораторная работа №1 «Анализ предметной области ИС»

Цель – исследование информационной системы и её объектов, для дальнейшего проектирования системы.

Этапы выполнения задания:

1. Описание общих сведений о предприятии: название, область деятельности, общие цели предприятия и функции.
2. Постановка общей задачи на автоматизацию (для определения масштаба проекта): разработка корпоративной системы, автоматизация деятельности отдельного отдела предприятия, разработка рабочего места специалиста.
3. Анализ и создание организационной структуры предприятия – описание иерархии подразделений, отделов, цехов, лабораторий, рабочих групп предприятия.
4. Создание структуры управления предприятия – директор, заместитель, начальник и т.п.

Варианты заданий:

1. Бюро по трудоустройству (кадровое агентство).
2. Гостиница.
3. Ломбард.
4. Интернет-магазин.
5. Ювелирная мастерская.
6. Библиотека.
7. Учет телефонных переговоров.
8. Туристическая фирма.
9. Распределение учебной нагрузки.
10. Фирма по продаже запчастей.
11. Прокат автомобилей.

Порядок выполнения работы

1. Изучить теоретическую часть лабораторной работы.
2. Построить организационную структуру предприятия и структуру управления предприятием в соответствии с заданием. Для построения использовать векторный графический редактор Microsoft Office Visio.
3. Проанализировать построенные схемы.
4. Оформить отчет.

Теоретические сведения

Предметная область — это часть реального мира, подлежащая изучению с целью создания базы данных для автоматизации процесса управления.

Анализ - определение того, что должна делать система.

Цель анализа при исследовании информационных систем - получение информации, необходимой для диагностики проблемы и исследования объекта и его системы, устранения проблем, наилучшего использования имеющихся возможностей. При таком анализе должны рассматриваться система и среда в их сопряженных состояниях, а также в системном единстве должны рассматриваться объект управления и система управления этим объектом.

К анализу предметной области при разработке АИС относятся:

- технико-экономическая характеристика предметной области (характеристика предприятия, краткая характеристика подразделения или видов его деятельности, экономическая сущность задачи, обоснование необходимости и цели использования вычислительной техники для решения задачи);

- постановка задачи (цель и назначение автоматизированного варианта решения задачи, общая характеристика организации решения задачи на ЭВМ, формализация расчетов);

- анализ существующих разработок и обоснование выбора технологии проектирования;

- обоснование проектных решений по видам обеспечения (по техническому обеспечению (ТО), по информационному обеспечению (ИО), по программному обеспечению (ПО)).

Разработчик, при необходимости, должен выполнить анализ области применения разрабатываемой системы с точки зрения определения требований к ней.

В качестве предметной области может выступать предприятие, фирма, объединение и т.д., или отдельный вид деятельности, протекающий в нем, поэтому в начале данного раздела необходимо отразить цель функционирования предприятия, его организационную структуру и основные параметры его функционирования и определить все основные виды деятельности.

При проведении анализа нужно дать общее описание рассматриваемых видов деятельности, а также характеристику технико-экономических свойств предприятия или организации как объекта управления.

Главными технико-экономическими свойствами объекта управления являются: цель и результаты деятельности, продукция и услуги, основные этапы и процессы рассматриваемой деятельности, используемые ресурсы. В ходе рассмотрения перечисленных свойств, для них, по возможности, следует указать количественно-стоимостные оценки и ограничения.

Поскольку объектом рассмотрения при разработке автономной задачи может служить какая-либо деятельность отдельного подразделения предприятия (например, отдела или цеха), его участка или отдельного сотрудника, то далее нужно привести краткую характеристику этого подразделения, в которой осуществляется рассматриваемая деятельность. Характеристика подразделения включает описание его структуры, перечень выполняемых в этом подразделении функций управления и его взаимодействие с другими подразделениями данного предприятия или подразделениями внешней среды.

Характеризуя подразделение предприятия, следует отразить особенности его функционирования, то есть принятые нормы и правила осуществления анализируемой деятельности, в условиях конкретной организации или предприятия. Итогом анализа предметной области является постановка задачи - формулировка задачи разработки проекта, выделение основных требований к проектируемой системе.

Технические требования к системе должны охватывать: функции и возможности системы; коммерческие и организационные требования; требования пользователя; требования безопасности и защиты; эргономические требования; требования к интерфейсам; эксплуатационные требования; требования к сопровождению; проектные ограничения и квалификационные требования. Технические требования к системе должны быть документально оформлены.

Пример анализа предметной области:

1. Общая характеристика предприятия

Анализ предметной области проектирования начинается с общего описания объекта проектирования.

ООО «Диванофф» - организация, осуществляющая предпринимательскую деятельность на территории города Владивостока на протяжении 8 лет.

Сфера деятельности предприятия – розничная торговля на рынке мягкой мебели. Ассортимент товаров включает в себя диваны, кресла, мягкие уголки разной комплектации, детскую мебель, кровати различного качества, ценовых категорий и конструкций. За время своей работы компания наладила крепкие деловые связи со многими поставщиками и напрямую с некоторыми заводами-производителями мебели.

Данная организация структурно состоит из отдела закупок, отдела продаж, склада и бухгалтерии (рисунок 1). Отдел продаж представляет собой торговую сеть, состоящую из трех магазинов-складов.

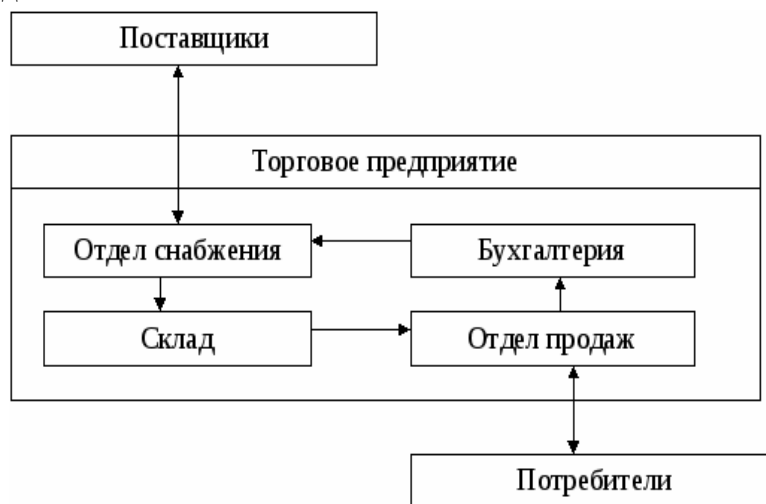


Рисунок 1 - Структурная схема компании

В основу организационной структуры ООО «Диванофф» положен функциональный принцип. Непосредственное руководство предприятием осуществляет генеральный директор. Управление отделами возложено на соответствующих руководителей:

1. Исполнительный директор;
2. Главный бухгалтер;
3. Начальник отдела снабжения;
4. Директора магазинов.

Представим организационную структуру управления предприятием на рисунке 2.



Рисунок 2 - Схема структуры управления ООО «Диванофф»

В рамках практической работы рассмотрим закупочную деятельность, функции которой выполняет отдел снабжения в лице начальника отдела и двух менеджеров по снабжению.

Контрольные вопросы к защите работы:

1. Что такое проектирование ПО.
2. Какие подходы к проектированию вам известны?
3. Сущность структурного подхода.
4. Принципы структурного подхода.
5. Что такое предметная область?
6. С какой функцией проводится анализ предметной области?

Лабораторная работа №2

«Применение структурного подхода в анализе требований и определении спецификаций ПО. Диаграмма переходов состояний».

Цель: построение диаграммы переходов состояний сложного объекта программной системы в анализе требований и определении спецификаций ПО.

Порядок выполнения работы

1. Изучить теоретическую часть лабораторной работы.
2. Построить диаграмму переходов состояний в соответствии с заданием. Для построения использовать векторный графический редактор Microsoft Office Visio.
3. Проанализировать построенную диаграмму.
4. Оформить отчет.

Теоретические сведения

1. Редактор Microsoft Office Visio.

Microsoft Office Visio - векторный графический редактор, редактор диаграмм и блок-схем для Windows. Выпускается в трёх редакциях: Standard, Professional и Pro for Office.

Аналогично с Adobe Reader, в стандартный набор программ MS Office входит только средство для просмотра и печати диаграмм Microsoft Visio Viewer. Полнофункциональная версия Microsoft Visio Professional для создания и редактирования монограмм и диаграмм в пакеты MS Office и входит и распространяется отдельно.

Первоначально Visio разрабатывался и выкупался компанией Visio Corporation. Microsoft приобрела компанию в 2000 году, тогда продукт назывался Visio 2000, был выполнен ребрендинг, и продукт был включен в состав Microsoft Office.

Файловые форматы

- VSD — диаграмма или схема,
- VSS — фигура,
- VST — шаблон,
- VDX — диаграмма в формате XML,
- VSX — фигура XML,
- VTX — шаблон XML,
- VSL — надстройка.
- VSDX — OPC/XML диаграмма,
- VSDM — OPC/XML диаграмма, содержащая макрос

Visio 2010 и более ранние версии Microsoft Visio поддерживают просмотр и сохранение диаграмм в форматах VSD и VDX. VSD является собственным бинарным файловым форматом, который используется во всех предыдущих версиях Visio. VDX является хорошо задокументированным XML "DatadiagramML" форматом. Начиная с версии Visio 2013, сохранение в формате VDX больше не поддерживается в пользу новых VSDX и VSDM файловых форматов. Созданные на основе стандарта Open Packaging

Conventions (OPC - ISO 29500, Часть 2), VSDX и VSDM файлы состоят из группы архивированных XML-файлов, находящихся внутри ZIP-архива. Единственная разница между VSDX и VSDM файлами состоит в том, что VSDM файл может содержать макросы. Из-за подверженности таких файлов макровирусам, программа обеспечивает строгую безопасность для них.

Visio 2010 и более ранние версии Microsoft Visio используют VSD формат как формат по умолчанию, Visio 2013 использует VSDX формат по умолчанию.

2. Диаграмма переходов состояний.

Характеристика диаграмм переходов состояний (SDT).

Диаграммы SDT демонстрируют поведение разрабатываемой программной системы при получении управляющих воздействий.

Под *управляющими воздействиями*, или *сигналами*, в данном случае понимают управляющую информацию, получаемую системой извне; например, управляющими воздействиями считают команды пользователя и сигналы датчиков, подключенных к компьютерной системе. Получив такое управляющее воздействие, разрабатываемая система должна выполнить определенные действия, а затем или остаться в том же состоянии, или перейти в другое состояние, зафиксировав некоторые изменения в системе.

Главное предназначение этой диаграммы — описать возможные последовательности состояний и переходов, которые в совокупности характеризуют поведение элемента модели в течение его жизненного цикла. Диаграмма переходов состояний представляет динамическое поведение сущностей на основе спецификации их реакции на восприятие некоторых конкретных событий.

Системы, которые реагируют на внешние воздействия других систем или пользователей, иногда называют реактивными. Если такие воздействия инициируются в произвольные случайные моменты времени, то говорят об асинхронном поведении модели.

Несмотря на то, что диаграммы чаще всего используются для описания поведения отдельных экземпляров классов (объектов), они также могут применяться для спецификации функциональности других компонентов моделей, таких как варианты использования, действующие лица, подсистемы, операции и методы.

Для построения диаграммы переходов состояний необходимо в соответствии с теорией конечных автоматов определить основные состояния, управляющие воздействия (или условия перехода), выполняемые действия и возможные переходы разрабатываемого программного обеспечения. Условные обозначения, используемые при построении диаграмм переходов состояний, показаны на рис. 2.3.



Рис. 2.3. Условные обозначения диаграмм переходов состояний:
а — начальное и конечное состояния; б — промежуточное состояние; в — переход

Диаграмма переходов состояний, по существу, представляет собой граф специального вида, который обозначает некоторый автомат. Понятие автомат в контексте UML обладает довольно специфической семантикой, основанной на теории автоматов.

Вершинами этого графа являются состояния и некоторые другие типы элементов автомата (псевдосостояния), которые изображаются соответствующими графическими символами. Дуги графа служат для обозначения переходов из состояния в состояние.

Простейшим примером визуального представления состояний и переходов на основе формализма автоматов может служить ситуация с исправностью технического устройства, такого как компьютер. В этом случае вводятся в рассмотрение два самых общих состояния: «исправен» и «неисправен» и два перехода: «выход из строя» и «ремонт». Графически эта информация может быть представлена в виде диаграммы состояний компьютера (рис. 2.4).



Рис. 2.4. Простейший пример диаграммы переходов состояний для технического устройства типа компьютер

Основными понятиями, входящими в формализм автомата, являются *состояние* и *переход*. Главное различие между ними в том, что время нахождения системы в отдельном состоянии существенно превышает время, которое затрачивается на переход из одного состояния в другое. Предполагается, что в пределе время перехода из одного состояния в другое равно нулю (если дополнительно ничего не сказано). Другими словами, переход объекта из состояния в состояние происходит мгновенно.

Для понимания семантики конкретной диаграммы состояний необходимо представлять не только особенности поведения моделируемой сущности, но и знать общие сведения по теории автоматов.

Сторожевое условие (guard condition), если оно есть, всегда записывается в прямых скобках после события-триггера и представляет собой некоторое булевское выражение. Напомним, что булевское выражение должно принимать одно из двух взаимно исключающих значений: «истина» или «ложь». Из контекста диаграммы состояний должна явно следовать семантика этого выражения.

Если сторожевое условие принимает значение «истина», то соответствующий переход может сработать, в результате чего объект перейдет в целевое состояние. Если же сторожевое условие принимает значение «ложь», то переход не может сработать и при отсутствии других переходов объект не может перейти в целевое состояние по этому переходу. Однако вычисление истинности сторожевого условия происходит только после возникновения связанного с ним события.

Для интерактивного программного обеспечения с развитым пользовательским интерфейсом основными управляющими воздействиями

выступают команды пользователя, для программного обеспечения реального времени — сигналы от датчиков и (или) оператора производственного процесса. Общим для этих типов программного обеспечения является наличие состояния ожидания, когда система

приостанавливает работу до получения очередного управляющего воздействия. Для интерактивного программного обеспечения наиболее характерно получение команд различных типов, а для программного обеспечения реального времени — однотипных сигналов либо от многих датчиков, либо требующих продолжительной обработки.

В отличие от интерактивных систем для систем реального времени обычно установлено более жесткое ограничение на время обработки полученного сигнала. Такое ограничение часто требует выполнения дополнительных исследований поведения системы во времени.

К программному обеспечению, при разработке которого требуется уточнение особенностей поведения посредством построения диаграммы переходов состояний, относится и программное обеспечение, ориентированное на работу в сети. При этом обычно отдельно строят модели поведения сервера и клиента, представляя сообщения, передаваемые между ними в виде управляющих воздействий.



Пример построения диаграммы переходов состояний.

На предлагаемой диаграмме (рис. 2.5) описываются возможные последовательности состояний и переходов, которые в совокупности характеризуют поведение объекта «Заказ» автоматизированной информационной системы «Склад оптовой торговли» в течение его существования (поступление, обработка, формирование поставки).

На ней отображаются функции, которые выполняются объектом «Заказ» в определенном состоянии. Метки деятельности имеют следующий синтаксис: *выполнить/ < деятельность >* (например, выполнить/проверить строку). Переходы имеют метки, которые синтаксически состоят из трех необязательных частей:

<Событие> *<[Условие]>* *</Действие>* (например, [не все строки проверены]/получить следующую строку).

Порядок оформления отчета по лабораторной работе

1. Введение.
Добавить теоретическую справку по теме.
2. Цель работы и вариант задания.
3. Пошаговый порядок выполнения работы с комментариями и снимками экрана работы, а также изображение готовой диаграммы с подробным описанием.

4. Вывод.

Контрольные вопросы к защите лабораторной работе:

1. Сущность структурного подхода.
2. Принципы структурного подхода.
3. Назовите виды диаграмм структурного подхода.
4. Цель построения диаграммы переходов состояний?
5. Характеристика диаграммы переходов состояний.
6. Условные обозначения и метки диаграммы переходов состояний: переход и состояние.
7. Что такое сторожевое условие?

Лабораторная работа №3

«Применение структурного подхода в анализе требований и определении спецификаций ПО. Функциональная диаграмма SADT».

Цель: построение функциональной диаграммы SADT объекта программной системы в анализе требований и определении спецификаций ПО.

Порядок выполнения работы

1. Изучить теоретическую часть лабораторной работы.
2. Построить диаграмму SADT, отражающую функциональную структуру объекта, в соответствии с заданием. Для построения использовать векторный графический редактор Microsoft Office Visio.
3. Проанализировать построенную диаграмму.
4. Оформить отчет.

Варианты заданий аналогичны.

Теоретические сведения

Функциональная диаграмма SADT

Характеристика функциональных диаграмм (SADT) — диаграммы SADT отражают взаимные связи функций разрабатываемого программного обеспечения. Они создаются на ранних стадиях проектирования систем, для того чтобы помочь проектировщику выявить основные функции и составные части проектируемой программной системы и, по возможности, обнаружить и устранить существенные ошибки. Для создания функциональных диаграмм предлагается использовать методологию SADT, предложенную Д.Россом. Применение методологии SADT позволяет построить модель, состоящую из диаграмм, фрагментов текстов и глоссария, имеющих ссылки друг на друга. Диаграммы — главные компоненты модели. Функции системы и интерфейсы представлены на диаграммах в виде блоков и дуг. Место соединения дуги с блоком определяет тип интерфейса. Управляющая информация входит в блок сверху, в то время как информация, которая подвергается обработке, показана с левой стороны блока, а результаты выхода даны с правой стороны.

Механизм (человек или автоматизированная система), осуществляющий операцию, представлен дугой, входящей в блок снизу (рис. 2.6).



Рис. 2.6. Функциональный блок и интерфейсные дуги

Одна из наиболее важных особенностей методологии SADT — постепенное введение все больших уровней детализации по мере создания диаграмм, отображающих

модель. Построение SADT - модели начинается с представления всей системы в виде простейшей компоненты — одного блока и дуг, изображающих интерфейсы с функциями вне системы. Поскольку единственный блок представляет всю систему как единое целое, имя, указанное в блоке, является общим. Это верно и для интерфейсных дуг — они также представляют собой полный набор внешних интерфейсов системы в целом.

Затем блок, в котором система дана в качестве единого модуля, детализируется на другой диаграмме с помощью нескольких блоков, соединенных интерфейсными дугами. Эти блоки представляют основные подфункции исходной функции. Данная декомпозиция выявляет полный набор подфункций, каждая из которых представлена как блок, границы которого определены интерфейсными дугами. Каждая из этих подфункций может быть декомпозирована подобным образом для более детального представления.

Во всех случаях каждая подфункция может содержать только те элементы, которые входят в исходную функцию. Кроме того, модель не может опустить какие-либо элементы, т. е., как уже отмечалось, родительский блок и его интерфейсы обеспечивают контекст. К нему нельзя ничего добавить, и из него не может быть ничего удалено.

Модель SADT представляет собой серию диаграмм с сопроводительной документацией, разбивающих сложный объект на составные части, представленные в виде блоков. Детали каждого из основных блоков показаны в виде блоков на других диаграммах.

Каждая детальная диаграмма является декомпозицией блока из более общей диаграммы. На каждом шаге декомпозиции более общая диаграмма называется *родительской* для более детальной диаграммы.

Дуги, входящие в блок и выходящие из него на диаграмме верхнего уровня, являются точно теми же самыми, что и дуги, входящие в диаграмму нижнего уровня и выходящие из нее, потому что блок и диаграмма представляют одну и ту же часть системы.

Стрелки, приходящие с родительской диаграммы или уходящие на нее, нумеруют, используя символы и числа. Символ обозначает тип связи: *I* — входные, *C* — управляющие, *M* — механизмы, *R* — результаты.

Число — номер связи по соответствующей стороне родительского блока, считая сверху вниз и слева направо. Все диаграммы связывают друг с другом иерархической нумерацией блоков: начальный уровень — АО, следующий — А1, А2 и т.п., следующий — А11, А12, А13 и т.д., где первая цифра — номер родительского блока, а последняя — номер конкретного субблока родительского блока. Детализацию завершают при получении функций, назначение которых хорошо понятно как заказчику, так и разработчику. Эти функции описывают, используя естественный язык или псевдокоды. В процессе построения иерархии диаграмм фиксируют всю уточняющую информацию и строят словарь данных, в котором определяют структуры и элементы данных, показанных на диаграммах. Таким образом, в результате получают спецификацию, которая состоит из иерархии функциональных диаграмм, описаний функций нижнего уровня и словаря, имеющих ссылки друг на друга.

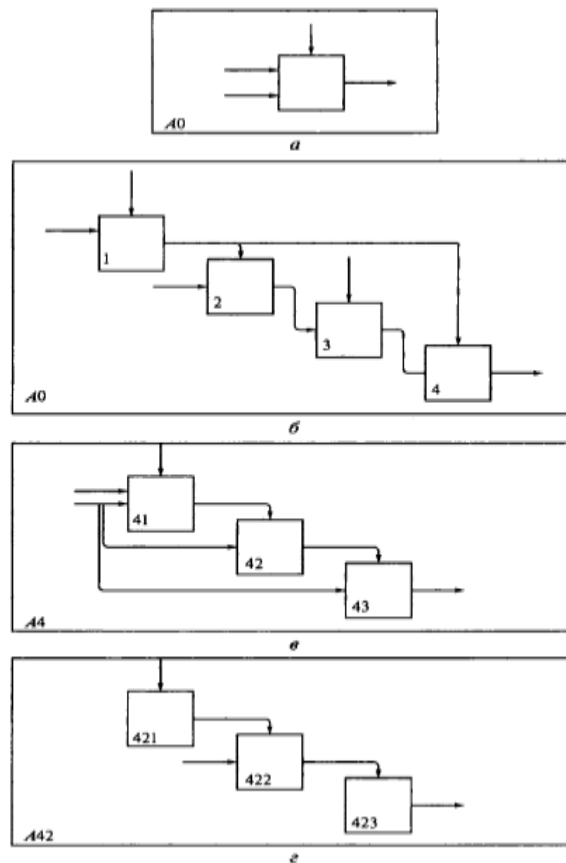


Рис. 2.7. Структура SADT-модели. Декомпозиция диаграмм

На рис. 2.7 представлена декомпозиция четырех диаграмм, показывающая структуру SADT-модели, общее (рис. 2.7, а) и детальное представление блока АО (рис. 2.7, б), декомпозицию блоков А4 (рис. 2.7, в) и А42 (рис. 2.7, г). Не присоединяемые дуги соответствуют входам, управлениям и выходам родительского блока.

Источник или получатель этих дуг может быть обнаружен только на родительской диаграмме. Не присоединяемые концы должны соответствовать дугам на исходной диаграмме. Все граничные дуги должны продолжаться на родительской диаграмме, чтобы она была полной и непротиворечивой.

Каждый компонент модели может быть декомпозирован на другой диаграмме, т. е. каждая диаграмма иллюстрирует «внутреннее строение блока на родительской диаграмме».

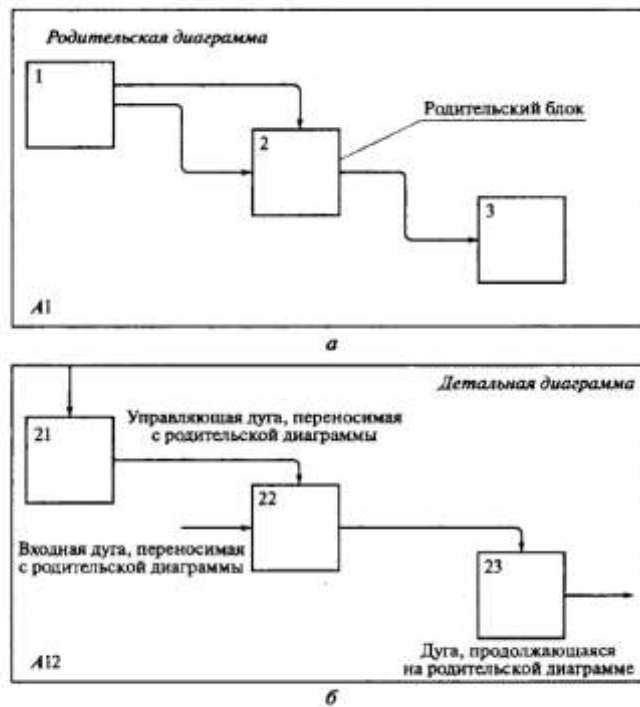


Рис. 2.8. Соответствие интерфейсных дуг родительской (а) и детальной (б) диаграмм

На рис. 2.8, а, б и рис. 2.9 представлены различные варианты выполнения функций и соединения дуг с блоками.

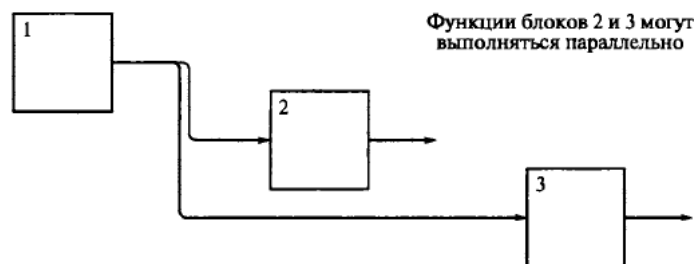


Рис. 2.9. Пример одновременного выполнения функций

На SADT-диаграммах не указаны явно ни последовательность, ни время. Обратные связи, итерации, продолжающиеся процессы и перекрывающиеся (по времени) функции могут быть изображены также с помощью дуг. Обратные связи могут выступать в виде комментариев, замечаний, исправлений и т.д. (рис. 2.10).

Одним из важных моментов при моделировании системы с помощью методологии SADT является точная согласованность типов связей между функциями. Различают, по крайней мере, следующие типы связей: случайная; логическая; временная; процедурная; коммуникационная; последовательная; функциональная.



Рис. 2.10. Пример обратной связи

Случайная связь возникает, когда конкретная связь между функциями мала или полностью отсутствует. Это относится к ситуации, когда имена данных на SADT-дугах в одной диаграмме имеют малую связь друг с другом.

Логическая связь — данные и функции собираются вместе, поскольку попадают в общий класс или набор элементов, но необходимых функциональных отношений между ними при этом не обнаруживается. Временная связь представляет функции, связанные во времени, когда данные используются одновременно или функции включаются параллельно, а не последовательно.

При процедурной связи функции сгруппированы вместе, поскольку они выполняются в течение одной и той же части цикла или процесса (рис. 2.11, а).

Диаграммы демонстрируют коммуникационную связь, когда блоки группируются вследствие того, что они используют одни и те же входные данные и (или) производят одни и те же выходные данные (рис. 2.11, б).

При последовательной связи выходные данные одной функции служат входными данными для другой функции. Связь между элементами на диаграмме является более тесной, чем на рассмотренных выше уровнях, поскольку моделируются причинно-следственные зависимости (рис. 2.11, в).

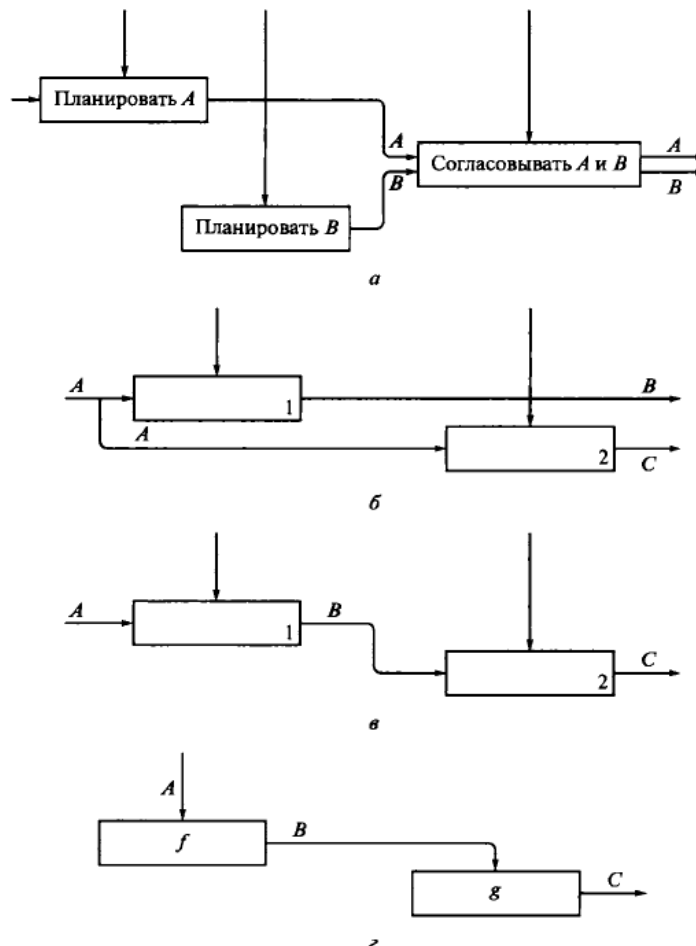


Рис. 2.11. Типы связей:

а — процедурная; б — коммуникационная; в — последовательная;
г — функциональная

Диаграмма отражает полную функциональную связь при наличии полной зависимости одной функции от другой. Диаграмма, которая является чисто функциональной, не содержит чужеродных элементов, относящихся к последовательному или более слабому типу связности. Одним из способов определения функционально

связанных диаграмм является рассмотрение двух блоков, связанных через управляющие дуги, как показано на рис. 2.11, г.

В математических терминах необходимое условие для простейшего типа функциональной связности (рис. 2.11, г) имеет следующий вид:

$$C = \partial(B) = g(f(A)).$$

Метод SADT может использоваться для моделирования самых разнообразных систем и для определения требований и функций.

В существующих системах метод SADT может применяться для анализа функций, выполняемых системой, и указания механизмов, посредством которых они осуществляются.

Пример разработки функциональной диаграммы программы построения графиков.

Разработку функциональных диаграмм продемонстрируем на примере уточнения спецификаций программы построения графиков и таблиц функций одной переменной.

На рис. 2.12, а показана диаграмма верхнего уровня, на которой хорошо видно, что является исходными данными для программы и получения каких результатов мы ожидаем.

Диаграмма на рис. 2.12, б уточняет функции программы. На ней показаны четыре блока: ввод— выбор и ее разбор, добавление функции в список, построение таблицы значений и построение графика функции. Для каждого блока определены исходные данные, управляющие воздействия и результаты. Согласно правилам обозначения входов — выходов, имеющих продолжение на родительской диаграмме, на данной диаграмме использованы следующие обозначения:

Л — функция; 12 — отрезок; 13 — шаг; C1 — вид график-таблица;

#1 — график функции на отрезке; R2 — таблица значений функции на отрезке.

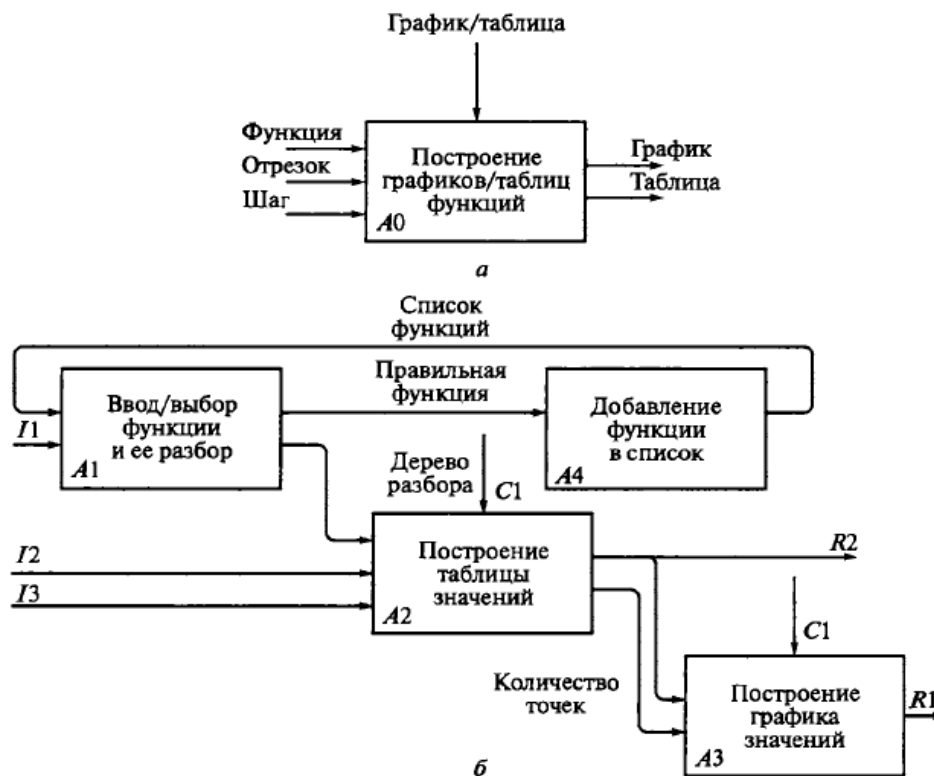


Рис. 2.12. Функциональные диаграммы для системы исследования функций:

а — диаграмма верхнего уровня; б — уточняющая диаграмма

Словарь в этом случае должен содержать описание всех данных, используемых в системе.

Функциональную модель целесообразно применять для определения спецификаций программного обеспечения, не предусматривающего работу со сложными структурами данных, так как она ориентирована на декомпозицию функций.

Контрольные вопросы к защите лабораторной работе:

1. Сущность структурного подхода.
2. Принципы структурного подхода.
3. Назовите виды диаграмм структурного подхода.
4. Цель построения функциональной диаграммы SADT?
5. Что такое декомпозиция?
6. Какие типы связей различают в диаграммы SADT?
7. Характеристика функциональной диаграммы SADT.
8. Условные обозначения и метки функциональной диаграммы SADT: функциональные блоки и интерфейсные дуги.
9. Что такое родительский блок?
10. Когда возникает случайная связь?
11. Что обозначает логическая связь?
12. Что представляет временная связь?
13. Что обозначает последовательная связь?

Лабораторная работа №4

«Применение структурного подхода в анализе требований и определении спецификаций ПО. Диаграмма потоков данных».

Цель: построение диаграммы потока данных информационной системы в анализе требований и определении спецификаций ПО.

Порядок выполнения работы

1. Изучить теоретическую часть лабораторной работы.
2. Построить диаграмму потока данных, являющуюся основным средством моделирования функциональных требований к проектируемой системе, в соответствии с заданием. Главная цель такого представления – продемонстрировать, как каждый процесс преобразует свои входные данные в выходные, а также выявить отношения между этими процессами.

Для построения использовать векторный графический редактор Microsoft Office Visio.

3. Проанализировать построенную диаграмму.
4. Оформить отчет.

Варианты заданий аналогичны.

Теоретические сведения

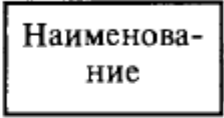
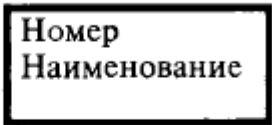
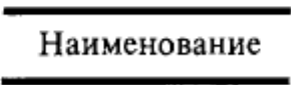
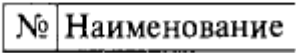
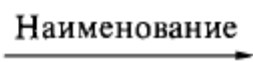
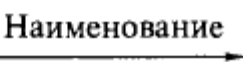
Диаграмма потока данных

Характеристика диаграмм потоков данных (DFD). Диаграмма DFD состоит из узлов обработки данных, средств их хранения и внешних по отношению к используемой диаграмме источников или потребителей данных.

Диаграмма потоков данных — основное средство моделирования функциональных требований к системе, проектируемой или реально существующей. В основе модели лежат понятия внешней сущности, процесса, хранилища (накопителя) данных потока данных.

Источники информации (внешние сущности) порождают информационные потоки (потоки данных), переносящие информацию к подсистемам или процессам; те, в свою очередь, преобразуют информацию и порождают новые потоки, которые переносят информацию к другим процессам или подсистемам, накопителям данных или внешним сущностям — потребителям информации.

Для изображения диаграмм потоков данных традиционно используют два вида нотаций — Йордана и Гейна—Сарсона, которые представлены в табл. 2.1.

Таблица 2.1. Виды нотаций		
Понятие	Нотация Йордана	Нотация Гейна — Сарсона
Внешняя сущность		
Процесс, система, подсистема		
Накопитель данных		
Поток данных		

Внешняя сущность — это материальный предмет или физическое лицо, представляющее собой источник или приемник информации, например, заказчики, персонал, поставщики, клиенты, склад. Определение некоторого объекта или системы в качестве внешней сущности указывает на то, что этот объект или эта система находятся за пределами границ анализируемой информационной системы. В процессе анализа некоторые внешние сущности

могут быть перенесены внутрь диаграммы анализируемой информационной системы, если это необходимо, или, наоборот, часть процессов информационной системы может быть вынесена за пределы диаграммы и представлена как внешняя сущность. При построении модели сложной информационной системы она может быть представлена в самом общем виде на так называемой *контекстной диаграмме*. На такой диаграмме показывают, как разрабатываемая система будет взаимодействовать с потребителями и источниками информации, т.е. описывают интерфейс системы с внешним миром. Система может быть представлена как единое целое либо может быть декомпозирована на ряд подсистем.

Номер подсистемы служит для ее идентификации. В поле имени вводится наименование подсистемы в виде предложения с подлежащим и соответствующими определениями и дополнениями. *Процесс* представляет собой преобразование входных потоков данных в выходные в соответствии с определенным алгоритмом.

Физически процесс может быть реализован различными способами: это может быть подразделение организации (отдел), выполняющее обработку входных документов и выпуск отчетов, программа, аппаратно реализованное логическое устройство и т.д. Номер процесса служит для его идентификации. В поле имени вводится наименование процесса, при этом использование таких глаголов, как обработать, модернизировать или отредактировать означает, как правило, недостаточно глубокое понимание данного процесса

и требует дальнейшего анализа. Информация в поле физической реализации показывает, какое подразделение организации, программа или аппаратное устройство выполняют данный процесс. *Накопитель данных* представляет собой абстрактное устройство для

хранения информации, которую можно в любой момент поместить в накопитель и через некоторое время извлечь; причем способы помещения и извлечения могут быть любыми. Накопитель

данных может быть реализован физически в виде ящика в картотеке, таблицы в оперативной памяти, файла на магнитном носителе и т.д.

В нотации Гейна—Сарсона накопитель данных идентифицируется буквой *D* и произвольным числом. Имя накопителя выбирается из соображения наибольшей информативности для проектировщика. Накопитель данных в общем случае выступает прообразом будущей базы данных, и описание хранящихся в нем данных должно быть увязано с информационной моделью.

Поток данных определяет информацию, передаваемую через некоторое соединение от источника к приемнику. Реальный поток данных может быть информацией, передаваемой по кабелю между двумя устройствами, пересылаемыми по почте письмами, магнитными лентами или дискетами, переносимыми с одного компьютера на другой, и т. д. Каждый поток данных имеет имя, отражающее его содержание.

Построение иерархии диаграмм потоков данных начинают с контекстной диаграммы, которая определяет наиболее общий вид системы. Обычно начальная контекстная диаграмма имеет форму звезды. Если проектируемая система содержит большое количество внешних сущностей (более 10), имеет распределенную природу или включает уже существующие подсистемы, то

строят иерархии контекстных диаграмм. В процессе детализации соблюдают правило балансировки — при детализации подсистемы могут использоваться компоненты только тех подсистем, с которыми у разрабатываемой подсистемы существует информационная связь (т.е. с которыми она связана потоками данных).

На недетализируемые процессы составляют спецификации, которые должны содержать описание функций данного процесса.

Такое описание может выполняться на естественном языке, с применением структурированного естественного языка (псевдокодов), с применением таблиц и деревьев решений, в виде схем алгоритмов.

Пример построения диаграммы потоков данных АИС «Склад оптовой торговли».

Построение иерархии диаграмм потоков данных начнем с контекстной диаграммы, которая определяет наиболее общий вид системы. Таким образом, определим, как разрабатываемая система будет взаимодействовать с приемниками и источниками информации (рис. 2.13).



Рис. 2.13. Начальная контекстная диаграмма (диаграмма нулевого уровня) в нотации Йордана для АИС «Склад оптовой торговли»

Автоматизированная информационная система «Склад оптовой торговли» предназначена для получения данных о движении и наличии товаров, приобретенных для оптовой продажи. Первичные документы по приходу товаров фиксируются в журнале поступления товаров. Оформление и учет реализации товаров зависят от способа расчета за приобретаемые товары между покупателем и продавцом. Менеджер склада ведет журнал учета закупок и отпуска товаров. Данные первичных документов сохраняются в соответствующих накопителях. Внешними сущностями для разрабатываемой системы являются поставщики, покупатели, менеджер склада, отдел учета и контроля, отдел приема и оформления заказов. Сведения о них сохраняются в соответствующих таблицах (справочниках). Поставщик передает товар на склад, документы на поставку товара (накладные, счета-фактуры) вводятся в базу данных. Покупатель подает заказ на приобретение товаров, в отделе приема и оформления заказов проверяется каждая строка поступившего заказа.

При отсутствии на складе какой-либо позиции оформляется заявка поставщику, т. е. инициируется поставка необходимого товара.

На основании заказа заполняются документы на реализацию товара, которые сохраняются в базе данных, распечатываются и выдаются покупателю. В конце каждого дня все первичные документы передаются менеджеру отдела учета (бухгалтеру). На основании сведений о приходе и реализации товара менеджеры склада и работники отдела учета формируют отчеты по оборотам и остаткам товара на складе.

Для построения диаграммы потоков данных нужно запустить MS Visio. Появится окно, в котором необходимо выбрать папку Shapes/Business Process. В открывшемся списке форм (Shapes) для построения диаграммы потоков данных следует выбрать пункт Data Flow Diagram Shapes. В результате проделанных действий на экране появится окно, в левой части которого будет отображен набор графических символов, а в правой части — лист для рисования диаграммы. Выбрать нужный компонент можно щелчком мыши по соответствующей пиктограмме, после чего графический символ переносится на рабочее поле мышкой при нажатой правой клавише. Диаграмма потоков данных АИС «Склад оптовой торговли» приведена на рис. 2.14.

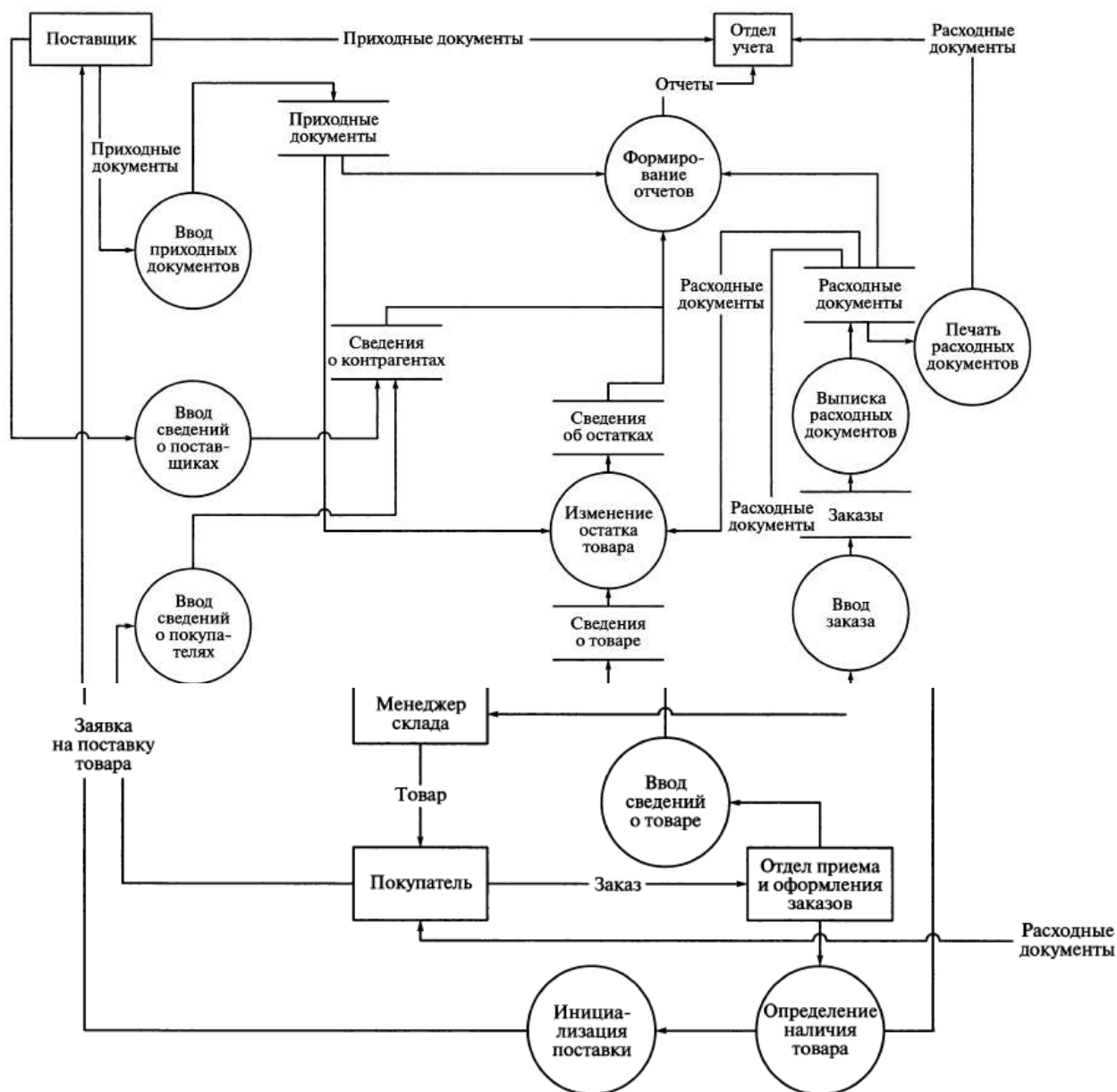


Рис. 2.14. Диаграмма потоков данных АИС «Склад оптовой торговли»

Пример построения диаграммы потоков данных подсистемы распределения студентов.

Разработаем диаграмму потоков данных автоматизированной системы, предназначенной для сбора и хранения информации о студентах, обучающихся на старших курсах учебного заведения. Система предназначена для использования в образовательных учреждениях, с целью предоставления

через Интернет потенциальным работодателям информации о выпускниках.

Начальная контекстная диаграмма потоков данных в нотации Гейна— Сарсона приведена на рис. 2.15. Внешними сущностями в ней являются работодатель, администратор и студент.

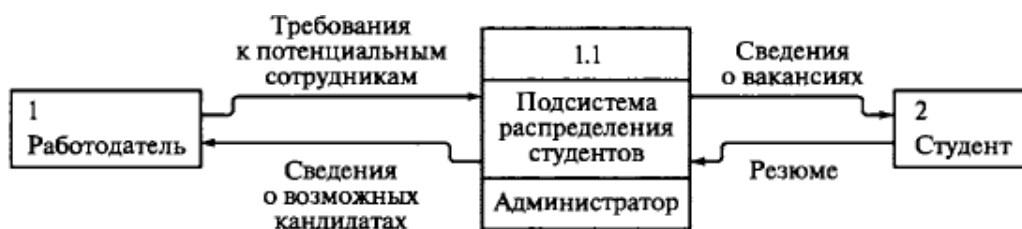


Рис. 2.15. Начальная контекстная диаграмма в нотации Гейна—Сарсона для системы распределения студентов

Работодатель регистрируется в системе, вводит данные для поиска кандидатов и получает от системы результаты поиска. Администратор вводит информацию о студентах, которые сохраняются в базе данных. Студент может изменить свою контактную информацию, изменения также сохраняются в базе данных. Выбранным студентам работодатель посылает сообщение с предложением о работе. Диаграмма потоков данных системы распределения студентов приведена на рис. 2.16.

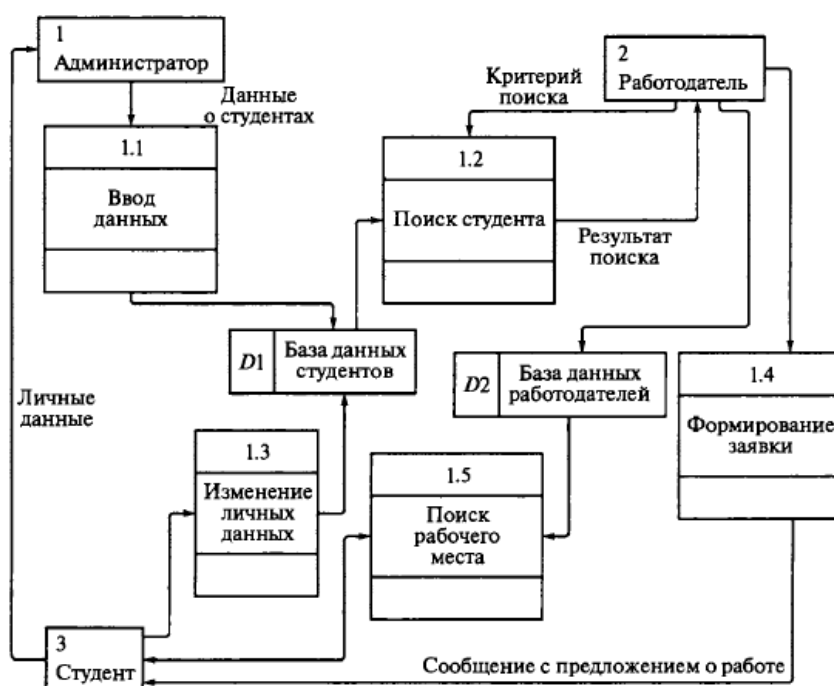


Рис. 2.16. Диаграмма потоков данных системы распределения студентов

Пример построения диаграммы потоков данных для системы учета успеваемости студентов.

Разработаем диаграммы потоков данных системы учета успеваемости студентов. В качестве внешних сущностей для системы выступают декан, заместитель декана по курсу и сотрудник деканата. Определим потоки данных между этими сущностями и системой.

Декан должен получать:

- сводку успеваемости по факультету (процент успеваемости групп, курсов и в целом по факультету) на текущий или указанный момент времени;
- полные сведения об учебе конкретного студента (успеваемость по всем изученным предметам всех завершенных семестров обучения с учетом пересдач).

Заместитель декана по курсу должен получать:

- сводку успеваемости по курсу (процент успеваемости по группам) на текущий или указанный момент;

- сведения о сдаче экзаменов и зачетов указанной группой;
- текущие сведения об успеваемости конкретного студента;
- полные сведения об учебе конкретного студента (успеваемость по всем изученным предметам всех завершенных семестров обучения с учетом пересдач);
- список задолжников по факультету и группам с указанием несданных предметов.

Сотрудник деканата должен обеспечивать:

- ввод списков студентов, зачисленных на первый курс;
- корректировку списков студентов в соответствии с приказами о зачислении, отчислении, переводе и т.п.;
- ввод учебных планов кафедр;
- ввод расписания сессии;
- ввод результатов сдачи зачетов и экзаменов на основании ведомостей и направлений.

Кроме того, сотрудник деканата должен иметь возможность получать:

- справку о прослушанных студентом предметах с указанием часов и итоговых оценок;
- приложение к диплому выпускника также с указанием часов и итоговых оценок.

В результате получаем контекстную диаграмму в нотации Гейна— Сарсона (рис. 2.17). Далее детализируем процессы в системе (рис. 2.18).



Рис. 2.17. Контекстная диаграмма системы учета успеваемости студентов

На детализирующей диаграмме потоков данных выделены две подсистемы: подсистема наполнения базы и подсистема формирования отчетов, а также хранилище данных, которое может быть реализовано как с помощью средств СУБД, так и без них.

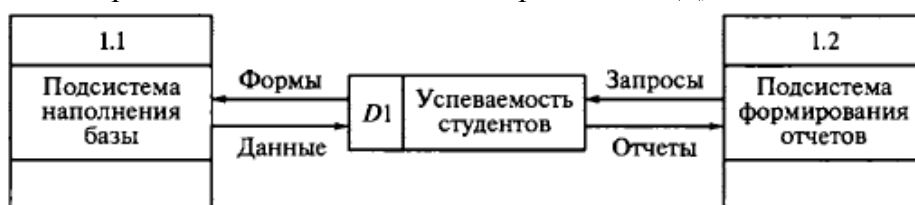


Рис. 2.18. Детализирующая диаграмма потоков данных второго уровня

Дальнейшая детализация процессов очевидна, однако становится ясно, что полная спецификация данной разработки должна включать описание базы данных в виде диаграммы «сущность—связь».

Порядок оформления отчета по лабораторной работе

1. Введение.
Добавить теоретическую справку по теме.
2. Цель работы и вариант задания.
3. Пошаговый порядок выполнения работы с комментариями и снимками экрана работы, а также изображение готовой диаграммы с подробным описанием.
4. Вывод.

Контрольные вопросы к защите лабораторной работе:

1. Сущность структурного подхода.
2. Принципы структурного подхода.
3. Назовите виды диаграмм структурного подхода.
4. Цель построения диаграммы потоков данных?
5. Характеристика диаграммы потоков данных.
6. Какие нотации используют для изображения диаграммы потоков данных?
7. Условные обозначения и метки диаграммы потоков данных:
Внешняя сущность; процесс, система, подсистема; накопитель данных; поток данных.
8. Что такое контекстная диаграмма?

Лабораторная работа №5

«Применение структурного подхода в анализе требований и определении спецификаций ПО. Диаграмма «Сущность-связь».

Цель: построение диаграммы «сущность-связь» информационной системы в анализе требований и определении спецификаций ПО.

Порядок выполнения работы

1. Изучить теоретическую часть лабораторной работы.
2. Построить диаграмму «сущность-связь», являющуюся основным средством моделирования концептуальной схемы базы данных в форме одной модели или нескольких локальных моделей, в соответствии с заданием. Данная диаграмма – ER-модель данных, обеспечивает стандартный способ определения данных и отношения между ними.

Для построения использовать векторный графический редактор Microsoft Office Visio.

3. Проанализировать построенную диаграмму.
4. Оформить отчет.

Варианты заданий аналогичны.

Теоретические сведения

Диаграмма «Сущность-связь»

Характеристика диаграмм «Сущность— связь». Данная диаграмма — (ER-модель данных) обеспечивает стандартный способ определения данных и отношений между ними. Она включает сущности и взаимосвязи, отражающие основные бизнес-правила предметной области. Диаграммы «сущность— связь» в отличие от функциональных диаграмм определяют спецификации структур данных программного обеспечения. Первый вариант модели «сущность — связь» был предложен П.Ченом. В дальнейшем многими авторами были разработаны свои варианты подобных моделей.

Все варианты диаграмм «сущность— связь» исходят из одной идеи — графическое изображение нагляднее текстового описания.

Все такие диаграммы используют графическое изображение сущностей предметной области, их свойств (атрибутов) и взаимосвязей между сущностями. Наиболее распространенной является нотация Баркера, ее и будем придерживаться.

Базовыми понятиями ER-модели данных (ER — Entity— Relationship) являются сущность, атрибут и связь.

Сущность — это класс однотипных реальных или абстрактных объектов (людей, событий, состояний, предметов и т.п.), информация о которых имеет существенное значение для рассматриваемой предметной области. Структурой данных называют совокупность правил и ограничений, которые отражают связи, существующие между отдельными частями (элементами) данных.

Каждая сущность должна иметь:

- уникальное имя;

- один или несколько атрибутов, которые либо принадлежат сущности, либо наследуются через связь;
- один или несколько атрибутов, которые однозначно идентифицируют каждый экземпляр сущности.

Экземпляр сущности — это конкретный представитель данной сущности. Имя сущности должно отражать тип или класс объекта, а не его конкретный экземпляр (студент, а не Иванов).



Рис. 2.19. Обозначения сущности в нотации Баркера:

а — без атрибутов; *б* — с указанием атрибутов; *в* — с уточнением атрибутов и их типов (# — ключевой, * — обязательный, ° — необязательный)

На диаграмме в нотации Баркера сущность изображается прямоугольником, иногда с закругленными углами (рис. 2.19, а). Каждая сущность обладает одним или несколькими атрибутами.

Атрибут — любая характеристика сущности, значимая для рассматриваемой предметной области и предназначенная для квалификации, идентификации, классификации, количественной характеристики или выражения состояния сущности (рис. 2.19, б).

Атрибут, таким образом, представляет собой некоторый тип характеристик или свойств, ассоциированных с множеством реальных или абстрактных объектов. Экземпляр атрибута — определенная характеристика конкретного экземпляра сущности. Атрибуты делятся на ключевые, т. е. входящие в состав уникального идентификатора ключа, и описательные — прочие.

Первичный ключ — это атрибут или совокупность атрибутов и (или) связей, предназначенная для уникальной идентификации каждого экземпляра сущности (совокупность признаков, позволяющих идентифицировать объект). Ключевые атрибуты помещают в начало списка и помечают символом «#» (рис. 2.19, в).

Описательные атрибуты могут быть обязательными или необязательными.

Обязательные атрибуты для каждой сущности всегда имеют конкретное значение, необязательные могут быть не определены.

Обязательные и необязательные описательные атрибуты помечают символами «*» и «°» соответственно.

Связь — это отношение одной сущности к другой или к самой себе. Каждая связь может иметь одну из двух модальностей связей.

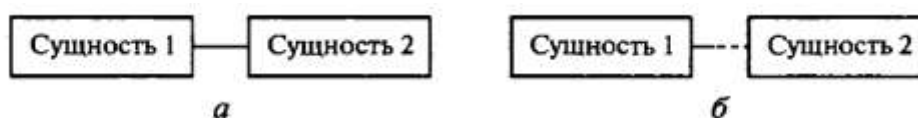


Рис. 2.20. Модальность связи:

а — обязательная; *б* — необязательная (пунктир до середины линии)

Если любой экземпляр одной сущности связан хотя бы с одним экземпляром другой сущности, то связь является обязательной (рис. 2.20, а). Необязательная связь представляет собой условное отношение между сущностями (рис. 2.20, б). Связь может иметь разную модальность с разных концов. Каждая связь может быть прочитана как слева направо, так и справа налево. Каждая сущность может обладать любым количеством связей с другими сущностями модели. Различают три типа отношений (рис. 2.21): «один-к-одному»; «один-ко-многим»; «многие-ко-многим».



Рис. 2.21. Обозначение отношений в нотации Баркера:

а — «один-к-одному»; *б* — «один-ко-многим»; *в* — «многие-ко-многим»

Связь «один-к-одному» означает, что один экземпляр первой сущности связан только с одним экземпляром второй сущности. Такая связь, скорее всего, свидетельствует о том, что была неверно разделена одна сущность на две (хотя иногда к такому типу связи прибегают в случае если есть необходимость «засекретить» часть данных).

При связи «один-ко-многим» каждый экземпляр первой сущности связан с несколькими экземплярами второй сущности. Связь «многие-ко-многим» показывает, что каждый экземпляр первой сущности может быть связан с несколькими экземплярами второй сущности, и наоборот. Такой тип связи является временным. Он допустим на ранних этапах разработки модели.

В дальнейшем такую связь необходимо заменить двумя связями «один-ко-многим» путем создания промежуточной сущности.

Независимая сущность представляет независимые данные, которые всегда присутствуют в системе. Они могут быть как связаны с другими сущностями, так и нет.

Зависимая сущность представляет данные, зависящие от других сущностей системы, поэтому она всегда должна быть связана с другими сущностями.

Ассоциированная сущность представляет данные, которые ассоциируются с отношениями между двумя и более сущностями.

Обычно данный вид сущностей появляется в модели для разрешения отношения «многие-ко-многим» (рис. 2.22).

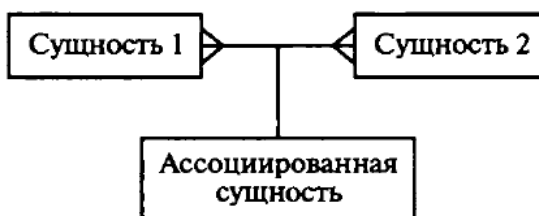


Рис. 2.22. Обозначение ассоциированной сущности в нотации Баркера

Если экземпляр сущности полностью идентифицируется своими ключевыми атрибутами, то говорят о полной идентификации сущности. В противном случае

идентификация сущности осуществляется с использованием атрибутов связанной сущности, что указывается черточкой на линии связи (рис. 2.23).

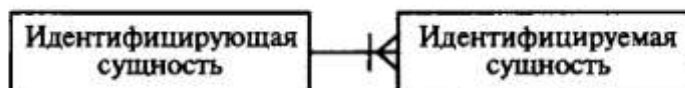
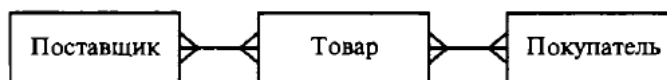


Рис. 2.23. Обозначение идентификации посредством другой сущности в нотации Баркера

Пример разработки диаграммы «сущность — связь» АИС «Склад оптовой торговли».

Основными сущностями для решения указанной задачи являются: поставщик, покупатель, товар.



а



б



в

Рис. 2.24. Варианты ER-диаграммы:

а — первый; б — промежуточный; в — окончательный

Сразу возникает очевидная связь между сущностями — «покупатели могут приобретать много товаров», «товары могут приобретаться многими покупателями». Отношение между ними относится к типу «многие-ко-многим» (рис. 2.24, а). Для разрешения этого отношения введем ассоциированную сущность «Накладная», которая отражает приобретение (продажу) товара покупателем (поставщиком) (рис. 2.24, б).

Проанализируем атрибуты сущностей. Каждый поставщик и покупатель является юридическим лицом и имеет наименование, адрес, банковские реквизиты. Каждый товар

имеет наименование, цену, характеризуется единицей измерения. Каждая накладная имеет уникальный номер, дату выписи, список товаров с количествами и ценами, а также общую сумму накладной. Покупатели покупают товары, получая при этом расходные накладные, в которые внесены данные о количестве и цене приобретенного товара. Каждый покупатель может получить несколько накладных.

Каждая накладная должна выписываться на одного покупателя. Каждая накладная должна содержать не менее одного товара (не может быть «пустой» накладной). Каждый товар, в свою очередь, может быть продан нескольким покупателям по нескольким накладным. Аналогичную цепь рассуждений можно выстроить для определения связей между сущностями «Товар» и «Поставщик».

Покупатель может быть одновременно и поставщиком, поэтому эти две сущности объединены в одну — «Контрагент». Теперь можно все это внести в диаграмму, как показано на рис. 2.24, в.

Уточненная диаграмма должна быть проверена с точки зрения возможности получения всех выходных данных (отчетов), указанных в техническом задании или показанных на диаграмме потоков данных разрабатываемой системы.

Порядок оформления отчета по лабораторной работе

1. Введение.
Добавить теоретическую справку по теме.
2. Цель работы и вариант задания.
3. Пошаговый порядок выполнения работы с комментариями и снимками экрана работы, а также изображение готовой диаграммы с подробным описанием.
4. Вывод.

32

Контрольные вопросы к защите лабораторной работе:

1. Сущность структурного подхода.
2. Принципы структурного подхода.
3. Назовите виды диаграмм структурного подхода.
4. Цель построения диаграммы «сущность-связь»?
5. Характеристика диаграммы «сущность-связь».
6. Основные понятия диаграммы «сущность-связь»:
сущность, экземпляр сущности, атрибут, первичный ключ, связь, независимая сущность.
7. Условные обозначения и метки диаграммы «сущность-связь»:
сущность без атрибутов, сущность с указанием атрибутов, сущность с уточнением атрибутов и их типов.
8. Назовите типы связей диаграммы «сущность-связь», как изображаются на диаграмме.
9. Что такое модальность связи и ее условное обозначение на диаграмме.
10. Что такое ассоциированная сущность и ее условное обозначение на диаграмме.

Лабораторная работа №6

«Проектирование ПО при структурном подходе. Структурная схема».

Цель: построение структурной схемы информационной системы в процессе проектирования ПО.

Порядок выполнения работы

1. Изучить теоретическую часть лабораторной работы.
2. Построить структурную схему, отражающую состав и взаимодействие по управлению частей разрабатываемого программного обеспечения, в соответствии с заданием. Для построения использовать векторный графический редактор Microsoft Office Visio.
3. Проанализировать построенную схему.
4. Оформить отчет.

Варианты заданий аналогичны.

Теоретические сведения

Структурная схема

Процесс проектирования программного обеспечения включает в себя определение структурных компонентов программной системы и связей между ними. Результат уточнения структуры может быть представлен в виде структурной схемы, которая дает достаточно полное представление о проектируемом программном обеспечении.

На рис. 3.1 приведена структурная схема программного обеспечения автоматизированной информационной системы «Склад оптовой торговли».



Рис. 3.1. Структурная схема программного обеспечения АИС «Склад оптовой торговли»

Процесс проектирования сложного программного обеспечения начинают с уточнения его структуры, т. е. определения структурных компонентов и связей между

ними. Результат уточнения структуры может быть представлен в виде структурной и/или функциональной схем и описания (спецификаций) компонентов.

Структурной называют схему, отражающую состав и взаимодействие по управлению частей разрабатываемого программного обеспечения.

Самый простой вид программного обеспечения - программа, которая в качестве структурных компонентов может включать только подпрограммы и библиотеки ресурсов.

Разработку структурной схемы программы обычно выполняют методом пошаговой детализации.

Структурными компонентами программной системы или программного комплекса могут служить программы, подсистемы, базы данных, библиотеки ресурсов и т. п.

Структурная схема программного комплекса демонстрирует передачу управления от программы-диспетчера соответствующей программе (рис. 3.1.1).



Рис. 3.1.1. Пример структурной схемы программного комплекса.

Структурная схема программной системы, как правило, показывает наличие подсистем или других структурных компонентов. В отличие от программного комплекса отдельные части (подсистемы) программной системы интенсивно обмениваются данными между собой и, возможно, с основной программой. Структурная же схема программной системы этого обычно не показывает (рис.3.1.2).



Рис.3.1.2. Пример структурной схемы программной системы учета успеваемости студентов

Более полное представление о проектируемом программном обеспечении с точки зрения взаимодействия его компонентов между собой и с внешней средой дает функциональная схема.

Порядок оформление отчета по лабораторной работе

1. Введение.
Добавить теоретическую справку по теме.
2. Цель работы и вариант задания.
3. Пошаговый порядок выполнения работы с комментариями и снимками экрана работы, а также изображение готовой схемы с подробным описанием.
4. Вывод.

Контрольные вопросы к защите лабораторной работе:

1. Сущность структурного подхода.
2. Принципы структурного подхода.
3. Что такое проектирование ПО.
4. Что включает процесс проектирования ПО.
5. Цель построения структурной схемы?
6. Каким методом выполняют разработку структурной схемы?

Лабораторная работа №7

«Проектирование ПО при структурном подходе. Функциональная схема».

Цель: построение функциональной схемы информационной системы в процессе проектирования ПО.

Порядок выполнения работы

1. Изучить теоретическую часть лабораторной работы.
2. Построить функциональную схему, отражающую состав и взаимодействие частей разрабатываемого программного обеспечения, в соответствии с заданием. Для построения использовать векторный графический редактор Microsoft Office Visio.
3. Проанализировать построенную схему.
4. Оформить отчет.

Варианты заданий аналогичны.

Теоретические сведения

Функциональная схема

Функциональная схема — это схема взаимодействия компонентов программного обеспечения с описанием информационных потоков, состава данных в потоках и указанием используемых файлов и устройств.

Схемы могут использоваться на различных уровнях детализации, при этом число уровней зависит от размеров и сложности задачи обработки данных. Уровень детализации должен быть таким, чтобы различные части и взаимосвязь между ними были понятны в целом.

Схемы данных отображают путь данных при решении задач и определяют этапы обработки, а также различные применяемые носители данных. Схема данных состоит из следующих символов:

- символы данных (символы данных могут также указывать вид носителя данных);
- символы процесса, производимого с данными (символы процесса могут также указывать функции, выполняемые вычислительной машиной);
- символов линий, указывающих потоки данных между процессами и (или) носителями данных;
- специальные символы, используемые для облегчения написания и чтения схемы.

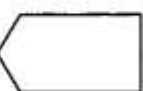
Схемы программ отображают последовательность операций в программе. Схема программы состоит из следующих символов:

- символы процесса, указывающие фактические операции обработки данных (включая символы, определяющие путь, которого следует придерживаться с учетом логических условий);
- линейные символы, указывающие поток управления;
- специальные символы, используемые для облегчения написания и чтения схемы.

Схемы работы системы отображают управление операциями и поток данных в системе. Схема работы системы состоит из следующих символов:

- символы данных, указывающие на наличие данных (символы данных могут также указывать вид носителя данных); символы процесса, указывающие операции, которые следует выполнить над данными, а также определяющие логический путь, которого следует придерживаться;
- линейные символы, указывающие потоки данных между процессами и (или) носителями данных, а также поток управления между процессами;
- специальные символы, используемые для облегчения написания и чтения блок-схемы.

Для изображения функциональных схем используют специальные обозначения, установленные ГОСТ 19.701—90 (табл. 3.1).

Таблица 3.1. Графические обозначения основных блоков алгоритмов		
Название	Графическое обозначение	Назначение
Терминатор		Символ отображает выход во внешнюю среду и вход из внешней среды (начало или конец схемы программы, внешнее использование и источник или пункт назначения данных)
Данные		Символ отображает данные, носитель данных не определен
Запоминающее устройство с прямым доступом		Символ отображает данные, хранящиеся в запоминающем устройстве с прямым доступом (магнитный диск, магнитный барабан, гибкий магнитный диск)
Документ		Символ отображает данные, представленные на носителе в удобочитаемой форме (машинограмма, документ для оптического или магнитного считывания, микрофильм, рулон ленты с итоговыми данными, бланки ввода данных)
Ручной ввод		Символ отображает данные, вводимые вручную во время обработки с устройств любого типа (клавиатура, переключатели, кнопки, световое перо, полосы со штриховым кодом)
Дисплей		Символ отображает данные, представленные в человекочитаемой форме на носителе в виде отображающего устройства (экран для визуального наблюдения, индикаторы ввода информации)
Процесс		Символ отображает функцию обработки данных любого вида (выполнение определенной опе-

Название	Графическое обозначение	Назначение
		рации или группы операций, приводящее к изменению значения, формы или размещения информации или к определению, по которому из нескольких направлений потока следует двигаться)
Предопределенный процесс		Символ отображает предопределенный процесс, состоящий из одной или нескольких операций или шагов программы, которые определены в другом месте (в подпрограмме, модуле)

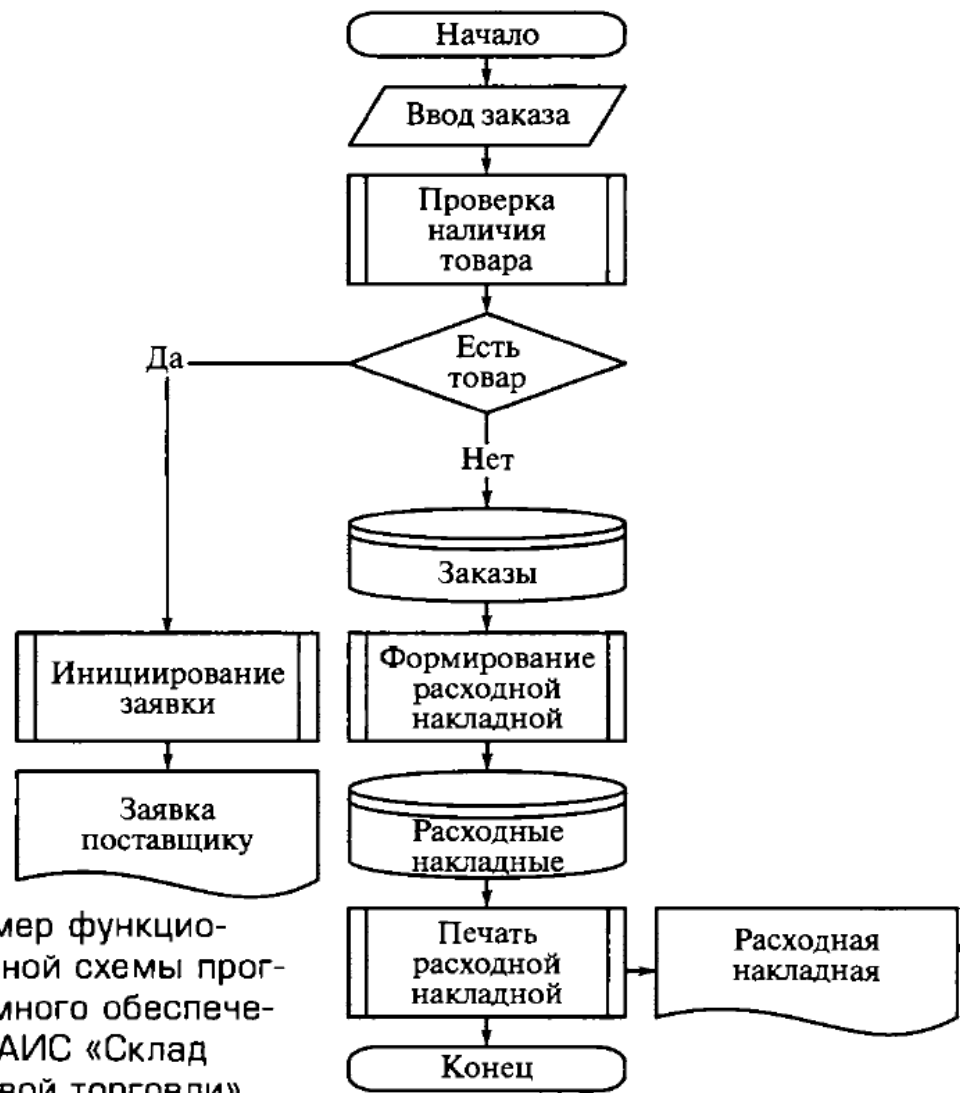


Рис. 3.2. Пример функциональной схемы программного обеспечения АИС «Склад оптовой торговли»

Функциональные схемы более информативны, чем структурные.

На рис. 3.2 приведена функциональная схема программной системы, реализующей операцию продажи товара автоматизированной информационной системы «Склад оптовой торговли».

Порядок оформления отчета по лабораторной работе

5. Введение.
Добавить теоретическую справку по теме.
6. Цель работы и вариант задания.
7. Пошаговый порядок выполнения работы с комментариями и снимками экрана работы, а также изображение готовой схемы с подробным описанием.
8. Вывод.

Контрольные вопросы к защите лабораторной работе:

1. Сущность структурного подхода.
2. Принципы структурного подхода.
3. Что такое проектирование ПО.
4. Что включает процесс проектирования ПО.
5. Цель построения функциональной схемы?
6. Каким методом выполняют разработку функциональной схемы?
7. Опишите основные элементы функциональных схем ПО. Какое графическое обозначение имеют эти блоки алгоритмов.
8. Что такое схема данных? Из каких символов состоит?
9. Что такое схема программ? Из каких символов состоит?
10. Что такое схема работы системы? Из каких символов состоит?

«Объектно-ориентированный подход к проектированию ПО»

Используя техническое задание из практической работы по дисциплине «Документирование и сертификация», провести анализ требований к программному обеспечению в соответствии с вариантом задания, применяя объектно-ориентированный подход и язык визуального моделирования UML. Для построения использовать case-средство Rational Rose.

Лабораторная работа №8

«Применение объектно-ориентированного подхода в анализе и проектировании ПО. Диаграмма вариантов использования».

Цель: построение диаграммы вариантов использования информационной системы в процессе проектирования ПО.

Порядок выполнения работы

1. Изучить теоретическую часть лабораторной работы.
1. Построить диаграмму вариантов использования, отражающую процедуры и задачи разрабатываемого программного обеспечения, в соответствии с заданием. Для построения использовать case-средство Rational Rose.
2. Проанализировать диаграмму.
3. Оформить отчет.

Варианты заданий аналогичны.

Теоретические сведения

В настоящее время существуют десятки приемов, методик, визуальных представлений, позволяющих моделировать требования к программному обеспечению. Необходимо определить целесообразность использования тех или иных приемов. Анализ требований должен соответствовать тому, что делает система, абстрагируясь от деталей реализации, т. е. от того, как она это делает.

Язык UML предоставляет в распоряжение пользователей легко воспринимаемый и выразительный язык визуального моделирования, предназначенный для разработки и документирования сложных моделей систем различного целевого назначения.

Определение вариантов использования. Разработку спецификаций программного обеспечения начинают с анализа требований к функциональности, указанных в техническом задании. В процессе анализа выявляют внешних пользователей разрабатываемого программного обеспечения и перечень отдельных аспектов его поведения в процессе взаимодействия с конкретными пользователями.

Аспекты поведения программного обеспечения были названы вариантами использования, или прецедентами (use cases).

Не следует путать вариант использования с конкретными операциями будущей системы. Каждый вариант использования связан с некоторой целью, имеющей самостоятельное значение; например, для текстового редактора Формирование оглавления — это вариант использования, а Связывание заголовков со специальными стилями — операция, которую необходимо выполнить, чтобы стало возможно автоматическое построение оглавления.

В зависимости от цели выполнения процедуры различают следующие варианты использования:

- **основные (базовые)** — обеспечивают требуемую функциональность разрабатываемого ПО;
- **вспомогательные** — обеспечивают выполнение необходимых настроек системы и ее обслуживание (например, архивирование информации и т.п.);
- **дополнительные** — обеспечивают дополнительные удобства для пользователя (как правило, реализуются, если не требуют серьезных затрат каких-либо ресурсов ни при разработке, ни при эксплуатации).

Графических средств языка UML на практике оказывается недостаточно для спецификации функциональных требований к программной системе. Следует отметить, что одним из требований языка UML является самодостаточность о моделях проектируемых систем. Однако изобразительных средств языка UML явно не хватает для того, чтобы учесть на диаграммах вариантов использования особенности функционального поведения сложной системы. С этой целью рекомендуется дополнять этот тип диаграмм текстовыми сценариями, которые уточняют или детализируют последовательность действий, совершаемых системой при выполнении ее вариантов использования.

В контексте языка UML сценарий используется для дополнительной иллюстрации взаимодействия актеров и вариантов использования. Предлагаются различные способы представления или написания подобных сценариев. Один из таких шаблонов представлен в табл. 4.1 и может быть рекомендован для применения на начальных этапах концептуального моделирования. В зависимости от уровня абстракции вариант использования может описываться кратко или более подробно. Краткая форма описания содержит: имя варианта использования, перечень действующих лиц (актеров) и тип варианта использования (основной, вспомогательный или дополнительный) и его краткое описание.

При написании сценариев вариантов использования важно понимать, что текст сценария должен дополнять или уточнять диаграмму вариантов использования, но не заменять ее полностью. В противном случае будут потеряны достоинства визуального представления моделей.

Основные понятия Диаграммы вариантов использования. Диаграммы вариантов использования позволяют наглядно представить ожидаемое поведение программной системы. Основными понятиями диаграмм вариантов использования являются: действующее лицо, вариант использования и связь.

На рис. 4.1 приведены условные обозначения, которые применяют при изображении диаграмм вариантов использования.

Таблица 4.1. Шаблон для написания сценария отдельного варианта использования			
Имя	Типичный ход событий, приводящий к успешному выполнению варианта использования	Исключение 1	Примечание 1
Действующие лица		Исключение 2	Примечание 2
Цель		Исключение 3	Примечание 3
Тип		...	...
Краткое описание			
Ссылки на другие варианты использования		Исключение л	Примечание л

Действующее лицо — внешняя по отношению к разрабатываемому программному обеспечению сущность, которая взаимодействует с ним в целях получения или предоставления какой-либо информации. Как уже упоминалось выше, действующими лицами могут быть пользователи, другое программное обеспечение или какие-либо технические средства.

Вариант использования — некоторая очевидная для действующего лица процедура, решающая его конкретную задачу. Все варианты использования так или иначе связаны с требованиями к функциональности разрабатываемой системы и могут сильно различаться по объему выполняемой работы (рис. 4.2).

Связь — взаимодействие действующих лиц и соответствующих вариантов использования.

Варианты использования также могут быть связаны между собой. При этом фиксируют связи использования и расширения.



Рис. 4.1. Основные условные обозначения диаграмм вариантов использования:

а — действующее лицо; б — вариант использования; в — связь

Использование (uses (include)) подразумевает, что существует некоторый фрагмент поведения разрабатываемого ПО, который повторяется в нескольких вариантах использования. Этот фрагмент оформляют как отдельный вариант и указывают связь с ним типа «использование».

Расширение (extends) применяют, если имеется два подобных варианта использования, различающиеся наличием в одном из них некоторых дополнительных действий. В этом случае дополнительные действия определяют как отдельный вариант использования, который связан с основным вариантом связью типа «расширение».

Главное назначение диаграммы вариантов использования заключается в формализации функциональных требований к системе и возможности согласования полученной модели с заказчиком на ранней стадии проектирования. Любой из вариантов использования может быть подвергнут дальнейшей декомпозиции на множество подвариантов использования отдельных элементов, которые образуют исходную сущность.

Пример разработки диаграммы вариантов использования.

Для иллюстрации особенностей спецификации функциональных требований на диаграмме вариантов использования можно рассмотреть модель системы «Склад оптовой торговли» (рис. 4.3).

Для первоначального понимания структуры программной системы выявляются действующие лица (люди-актеры или системы, между которыми происходит взаимодействие). Рассматриваемая система имеет пять актеров, двое из которых выступают контрагентами, а другие — менеджерами склада, осуществляющими выполнение всех операций. Каждый из этих актеров взаимодействует с системой, хотя главными актерами, пожалуй, являются поставщики и покупатели (контрагенты), поскольку именно они инициируют функциональность системы. Далее формулируются

варианты использования, т. е. действия, выполняемые системой для реализации общения действующих лиц (актеров).

Каждый из действующих лиц преследует определенные цели по отношению к системе: поставщик — сдать товар на склад, покупатель — приобрести товар, менеджер склада — принять и отпустить товар, менеджер учетного отдела — определить объемы поступления и продаж и проанализировать товарный запас. На основании этих целей можно сформулировать базовые варианты использования и проанализировать взаимосвязи между ними. На самом деле вариантов использования может быть гораздо больше.

Например, проверить платежеспособность клиента, получить информацию о товаре, оценить запасы товара на складе, получить оплату и т. д. Однако эта диаграмма дает понять, что будет делать система, как она будет функционировать.



Рис. 4.2. Пример диаграммы вариантов использования для тестовой системы

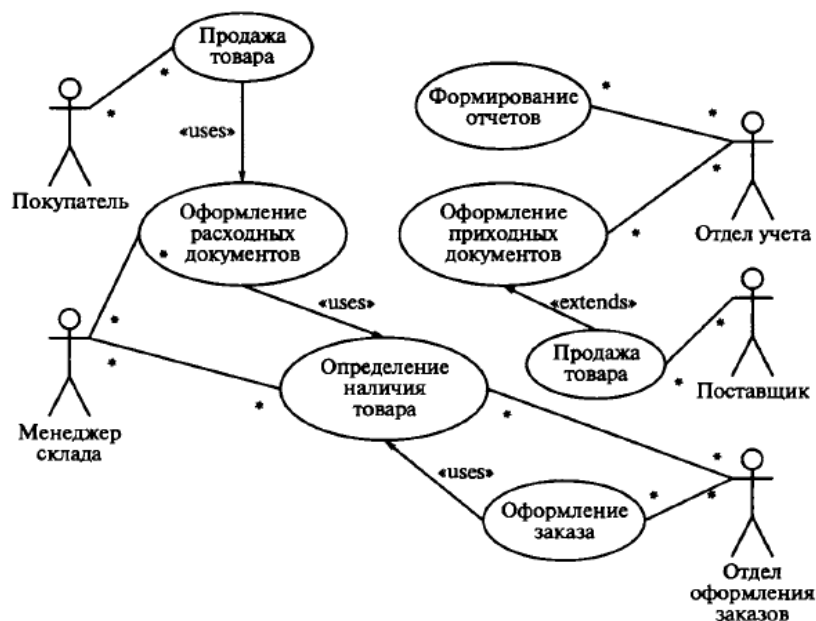


Рис. 4.3. Диаграмма вариантов использования для проектирования программного обеспечения АИС «Склад оптовой торговли»

На следующем этапе разработки модели вариантов использования для рассматриваемой системы следует дополнить данную диаграмму текстовым сценарием, написанным на основе предложенного ранее шаблона. Этот сценарий будет дополнять

диаграмму, раскрывая содержание и логическую последовательность отдельных действий, которые выполняются системой и актерами в процессе поступления и реализации товаров. В этом случае сценарий удобно представить в виде таблиц, каждая из которых описывает отдельный раздел шаблона.

В главном разделе сценария указывается имя рассматриваемого варианта использования, имена взаимосвязанных с ним актеров, цель выполнения варианта, условный тип и ссылки на другие варианты использования (табл. 4.2).

В следующем разделе сценария описывается последовательность действий, приводящая к успешному выполнению рассматриваемого варианта использования. При этом инициатором действий должен выступать актер Покупатель. Для удобства последующих ссылок каждое действие актеров помечается порядковым номером в последовательности действий (табл. 4.3).

Таблица 4.2. Сценарий варианта использования	
Вариант использования	Продажа товара
Актеры	Покупатель, Менеджер отдела оформления заказов, Менеджер склада
Краткое описание	Покупатель запрашивает товар. Менеджер отдела оформления заказов резервирует товар, оформляет заказ, передает заказ Менеджеру склада. Покупатель оплачивает товар, получает товар на складе
Цель	Получение необходимого товара
Тип	Базовый
Ссылки на другие варианты использования	Включает в себя варианты использования: определить наличие товара; оформить заказ

Таблица 4.3. Последовательность действий актеров	
Действия актеров	Отклик системы
1. Покупатель запрашивает товар Исключение 1. На складе нет необходимого количества запрашиваемого товара	2. Менеджер отдела оформления заказов проверяет наличие необходимого товара на складе 3. Менеджер отдела оформления заказов резервирует нужный товар
4. Покупатель оплачивает товар Исключение 2. Покупатель не оплатил товар	5. Менеджер отдела оформления заказов выдает разрешение на получение товара 6. Менеджер отдела оформления заказов передает заказ на склад 7. Менеджер склада выдает товар и расходную накладную покупателю 8. Менеджер оформления заказов блокирует получение товара покупателем

В третьем разделе сценария описывается последовательность действий, выполняемых при возникновении исключительных ситуаций, или исключений (табл. 4.4).

Можно дополнить данный сценарий, аналогичным образом описав не только варианты использования «Оформление заказа» и «Определение наличия товара», но и рассмотрев другие исключения, например оформление скидок постоянным покупателям и т. п. При этом полнота сценариев и модели вариантов использования будут определяться теми функциональными требованиями, которые сформулированы в рамках конкретного проекта.

Отдельные небольшие по своему объему сценарии могут быть размещены на диаграмме в форме примечаний *note*, предназначенных для включения в модель произвольной текстовой информации, которая имеет непосредственное отношение к контексту разрабатываемого проекта. В качестве такой информации могут быть комментарии разработчика (например, дата и версия разработки диаграммы или ее отдельных компонентов), ограничения (например, на значения отдельных связей или экземпляры сущностей) и помеченные значения. Применительно к диаграммам вариантов использования примечание может иметь уточняющую информацию, относящуюся к контексту тех или иных вариантов использования.

Таблица 4.4. Последовательность действий актеров при возникновении исключительных ситуаций	
Действия актеров	Отклик системы
Исключение 1. На складе нет необходимого количества запрашиваемого товара	
4. Покупатель оплачивает товар	3. Менеджер отдела оформления заказов инициирует поставку нужного товара
Исключение 2. Покупатель не оплатил товар	
	8. Менеджер оформления заказов блокирует получение товара покупателем

Графически примечания на всех типах диаграмм обозначаются прямоугольником с «загнутым» верхним правым уголком (рис. 4.4).

Собственно текст примечания размещается внутри этого прямоугольника. Примечание может относиться к любому элементу диаграммы, в этом случае их соединяет пунктирная линия. Если примечание относится к нескольким элементам, то от него проводятся, соответственно, несколько линий. Как уже отмечалось, примечания могут присутствовать не только на диаграмме вариантов использования, но и на других канонических диаграммах.

Рекомендации к разработке диаграмм вариантов использования.

Как было отмечено ранее, одно из главных назначений диаграммы вариантов использования заключается в формализации функциональных требований к системе. Диаграмма вариантов использования может служить основой для согласования с заказчиком функциональных требований к системе на ранней стадии проектирования.

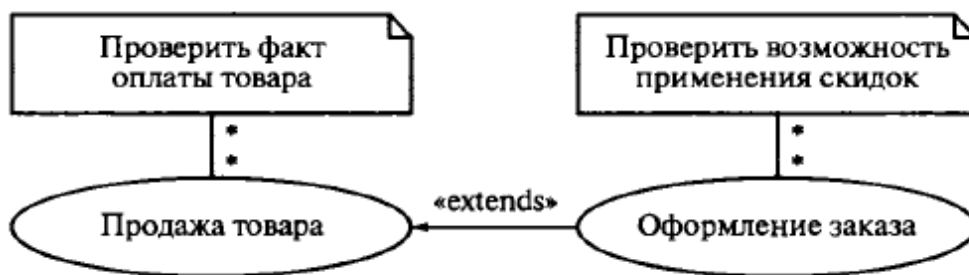


Рис. 4.4. Пример примечаний на диаграммах вариантов использования

Любой из базовых вариантов использования в последующем может быть подвергнут декомпозиции на частные варианты использования. При этом рекомендуется, чтобы общее количество актеров в модели не превышало 20, а вариантов использования — 50. В противном случае модель теряет свою наглядность и, возможно, заменяет собой одну из некоторых других диаграмм.

Для разработки диаграммы вариантов использования рекомендуется соблюдать некоторую последовательность действий:

- определить главных, или первичных, и второстепенных актеров;
- определить цели главных актеров по отношению к системе;
- сформулировать основные варианты использования, которые специфицируют функциональные требования к системе;
- упорядочить варианты использования по степени убывания риска их реализации;
- рассмотреть все базовые варианты использования в порядке убывания их степени риска;
- выделить участников, интересы, предусловия и постусловия выполнения выбранного варианта использования;
- написать успешный сценарий реализации выбранного варианта использования;
- определить исключения или неуспех в выполнении сценария варианта использования;
- написать сценарии для всех исключений;
- выделить общие варианты использования и изобразить их взаимосвязи с базовыми со стереотипом *uses (include)*;
- выделить варианты использования для исключений и изобразить их взаимосвязи с базовыми со стереотипом *extend*;
- проверить диаграмму на отсутствие дублирования вариантов использования и актеров.

Семантика построения диаграммы вариантов использования должна определяться следующими особенностями рассмотренных выше элементов модели. Отдельный экземпляр варианта использования по своему содержанию является выполнением последовательности действий, которая инициализируется посредством экземпляра сообщения от экземпляра актера. В качестве отклика или ответной реакции на сообщение актера выполняется последовательность действий, установленная для данного варианта использования.

При этом актеры могут генерировать новые сообщения для инициирования вариантов использования. Подобное взаимодействие будет продолжаться до тех пор, пока не закончится выполнение требуемой последовательности действий экземпляром варианта использования и указанный в модели экземпляр актера не получит требуемый экземпляр сервиса.

Окончание взаимодействия означает отсутствие инициализации сообщений от актеров для базовых вариантов использования.

Варианты использования могут быть дополнительно специфицированы примечаниями с текстом, которые в последующем могут стать прототипами операций и методов совместно с атрибутами.

Дальнейшая разработка моделей связана с реализацией вариантов использования в виде графа деятельности посредством конечного автомата или любого другого механизма логического представления поведения, включающего предусловия и постусловия. Взаимодействие между вариантами использования и актерами может уточняться на диаграмме кооперации, когда описываются взаимосвязи между системой, содержащей эти варианты использования, и окружением или внешней средой этой системы.

Порядок оформления отчета по лабораторной работе

1. Введение.
Добавить теоретическую справку по теме.
2. Цель работы и вариант задания.
3. Пошаговый порядок выполнения работы с комментариями и снимками экрана работы, а также изображение готовой схемы с подробным описанием.
4. Вывод.

Контрольные вопросы к защите работы:

1. Сущность объектно-ориентированного подхода.
2. Принципы объектно-ориентированного подхода.
3. Что такое проектирование ПО?
4. Что включает процесс проектирования ПО?
5. Охарактеризуйте понятие UML.
6. Каковы преимущества использования UML?
7. Перечислите виды диаграмм в UML.
8. Цель построения диаграммы вариантов использования?
9. Опишите основные элементы диаграммы вариантов использования. Какое графическое обозначение имеют элементы.
10. Что такое действующее лицо?
11. Что такое вариант использования?
12. Какие типы связей в этой диаграмме вам известны?

Лабораторная работа №9

«Применение объектно-ориентированного подхода в анализе и проектировании ПО. Диаграмма деятельности».

Цель: построение диаграммы деятельности информационной системы в процессе проектирования ПО.

Порядок выполнения работы

1. Изучить теоретическую часть лабораторной работы.
2. Построить диаграмму деятельности, отражающую подробное описание каждой задачи разрабатываемого программного обеспечения, в соответствии с заданием. Для построения использовать case-средство Rational Rose.
3. Проанализировать построенную схему.
4. Оформить отчет.

Варианты заданий аналогичны.

Теоретические сведения

Характеристика диаграмм деятельности. Если диаграмма вариантов использования дает «вид сверху» на функциональность системы, диаграмма деятельности, напротив, позволяет подробно иллюстрировать отдельный вариант использования и его сценарии.

В зависимости от степени детализации диаграммы деятельности могут использоваться на разных этапах разработки. На этапе анализа требований и уточнения спецификаций диаграммы деятельности позволяют конкретизировать основные функции разрабатываемого программного обеспечения.

Под деятельностью в данном случае понимают задачу (операцию), которую необходимо выполнить вручную или с помощью средств автоматизации. Каждому варианту использования соответствует своя последовательность задач. В теоретическом плане диаграмма деятельности — это обобщенное представление алгоритма, реализующего анализируемый вариант использования. На диаграмме деятельность обозначается прямоугольником с закругленными углами (рис. 4.5, а). Диаграммы деятельности позволяют описывать альтернативные и параллельные процессы.



Рис. 4.5. Условные обозначения диаграммы деятельности:

а — деятельность; б — выбор; в — линейки синхронизации; г — начало; д — конец

Для обозначения альтернативных процессов используют ромб (рис. 4.5, б), условие указывают рядом, а альтернативы «да», «нет» — рядом с соответствующими выходами. С помощью этого же блока можно построить циклический процесс. Множественность активации деятельности обозначают символом «*», помещенным рядом со стрелкой

активации деятельности, и при необходимости уточняют надписью вида «для каждой строки».

Для обозначения параллельных процессов используют линейки синхронизации (рис. 4.5, в). Условные обозначения начала и окончания диаграммы деятельности даны на рис. 4.5, г и д. Пример диаграммы деятельности с указанием параллельности процессов приведен на рис. 4.6. Условие синхронизации можно уточнить, указав его на диаграмме (рис. 4.7).

Пример построения диаграммы деятельности.

В предыдущем примере для конкретизации описания вариантов использования в автоматизированной информационной системе «Склад оптовой торговли» был разработан текстовый сценарий. Этот сценарий дополняет диаграмму, раскрывая содержание отдельных действий, выполняемых системой и актерами. Однако вместо описания вариантов использования или в дополнение к ним можно использовать диаграммы деятельности. Диаграмма деятельности позволяет проиллюстрировать вариант использования с требуемой степенью подробности. Имеет смысл уточнять только те варианты использования, краткое описание которых недостаточно для понимания сущности решаемых проблем.

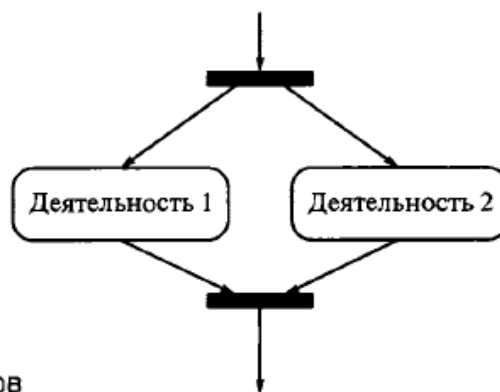


Рис. 4.6. Пример диаграммы деятельности с указанием параллельности процессов

В рамках разрабатываемой модели построим диаграммы деятельности для реализации вариантов использования «Поставка товара» (рис. 4.8) и «Продажа товара» (рис. 4.9). Диаграмма деятельности позволяет проиллюстрировать вариант использования с различной степенью подробности. Полная модель системы может содержать несколько диаграмм деятельности, каждая из которых описывает последовательность реализации либо наиболее важных вариантов использования (типичный ход событий и все исключения), либо нетривиальных операций классов.

Помимо стандартного формата описания, UML предлагает вариант с «плавательными дорожками». Этот формат удобен для описания случая, когда в варианте использования участвуют несколько действующих лиц. При этом все состояния действия на диаграмме деятельности делятся на отдельные группы, которые отделяются друг от друга вертикальными линиями (рис. 4.10).

Применение дорожек открывает дополнительные возможности для наглядного представления бизнес-процессов, позволяя специфицировать деятельность подразделений предприятий (рис. 4.11).

В случае типового проекта большинство деталей реализации действий могут быть известны заранее на основе анализа существующих систем или предшествующего опыта

разработки систем прототипов. Использование типовых решений может существенно сократить время разработки и избежать возможных ошибок при реализации проекта.

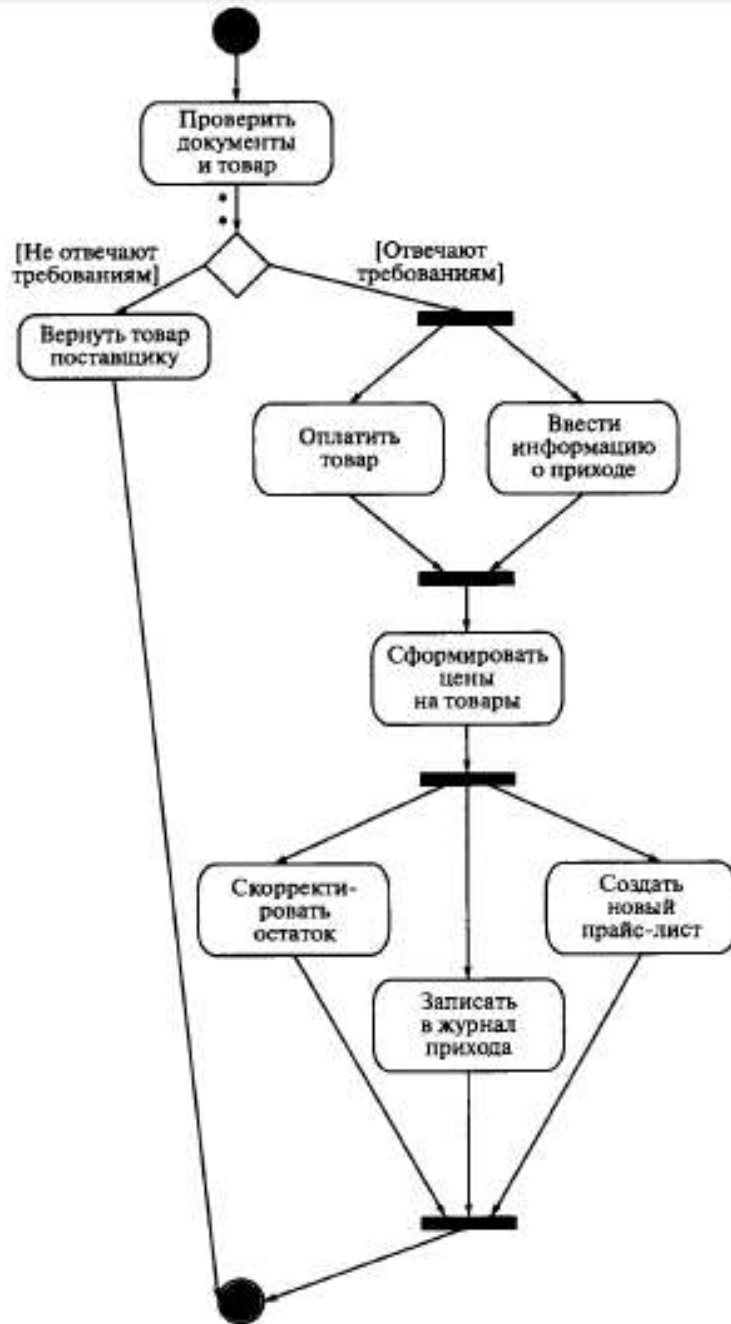
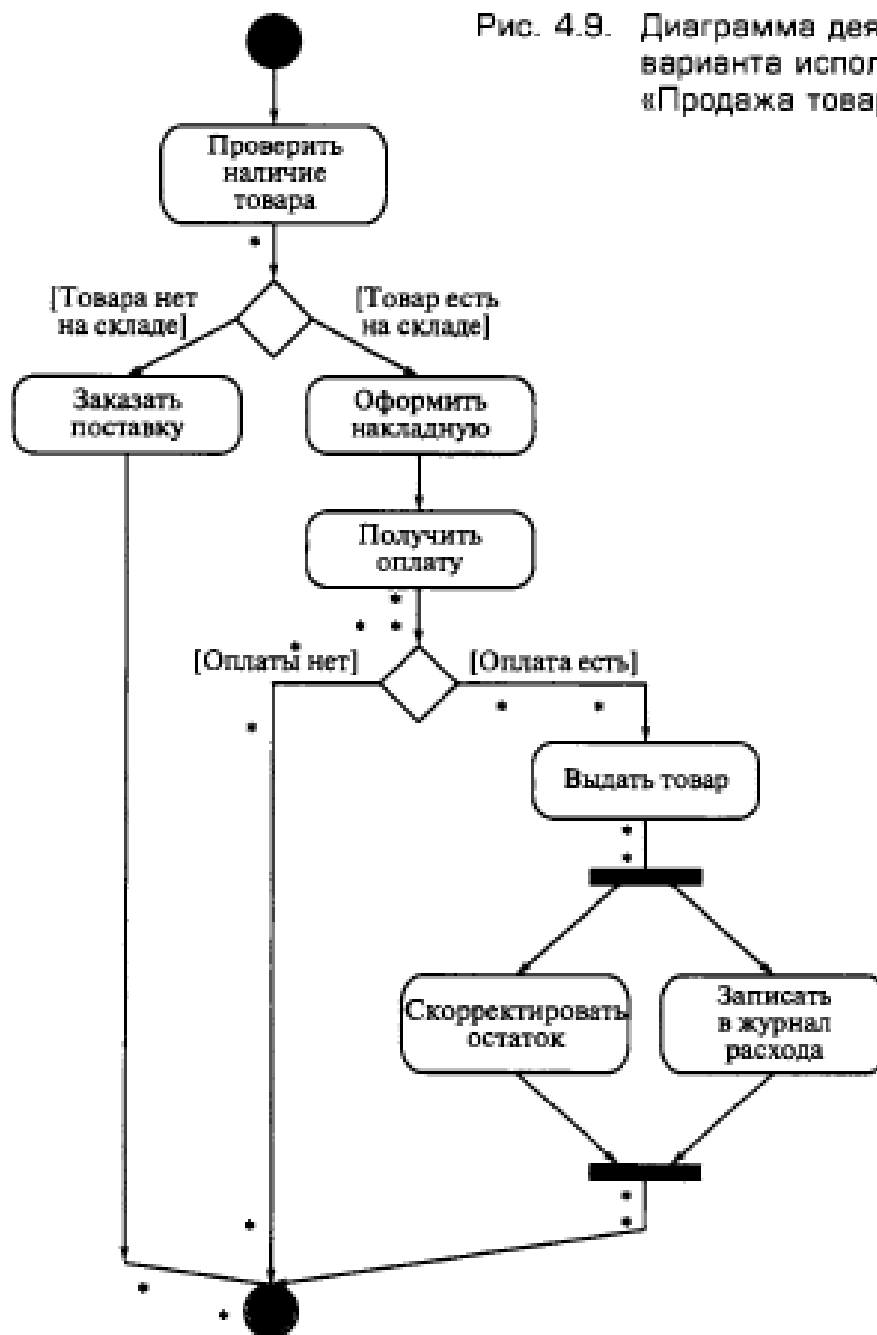


Рис. 4.8. Диаграмма деятельности для варианта использования «Поставка товара»

Рис. 4.9. Диаграмма деятельности для варианта использования «Продажа товара»



51

Рис. 4.10. Вариант диаграммы деятельности с дорожками

Объект 1	Объект 2	Объект 3

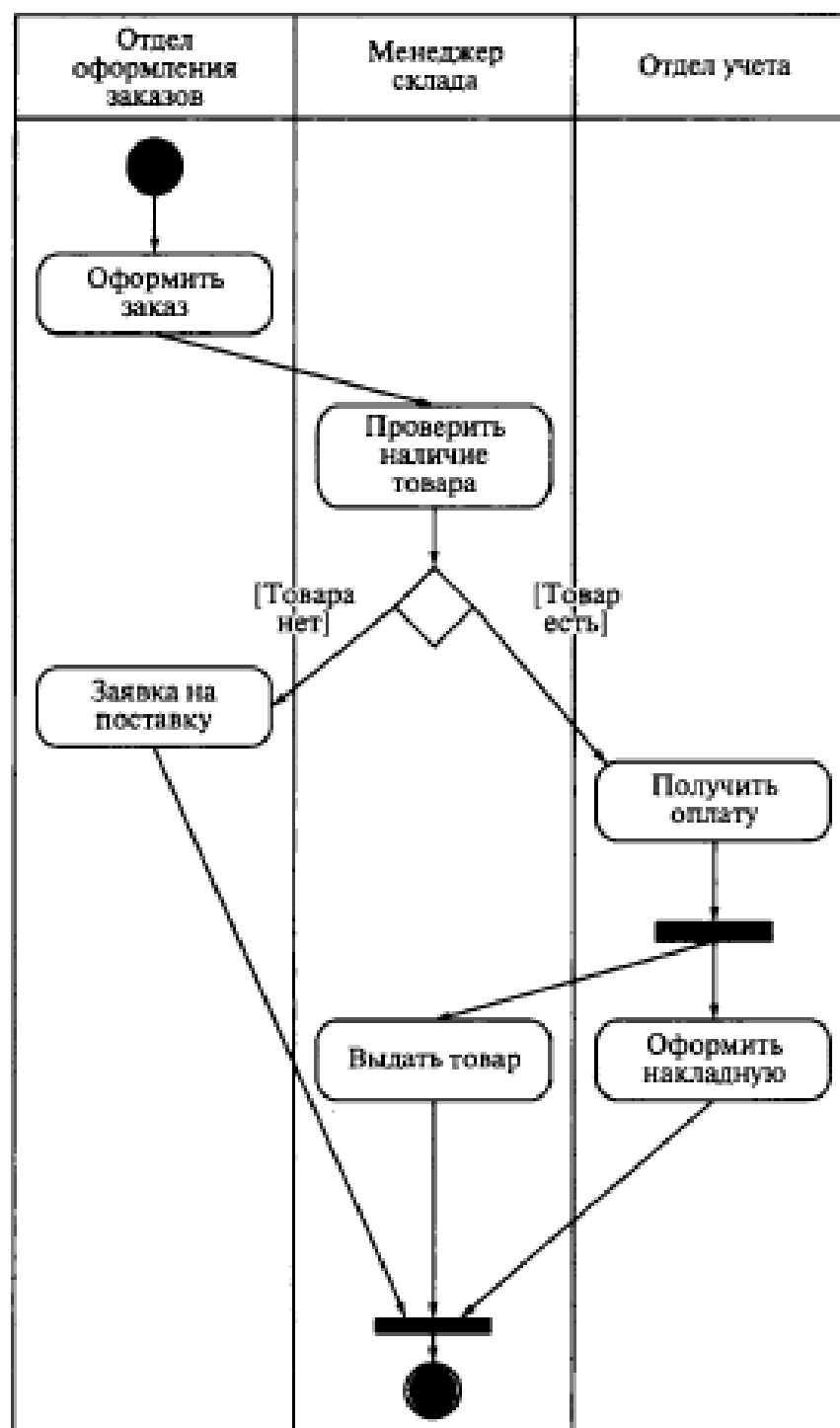


Рис. 4.11. Пример применения диаграммы деятельности с дорожками для варианта использования «Оформление заказа»

Порядок оформления отчета по лабораторной работе

1. Введение.
Добавить теоретическую справку по теме.
2. Цель работы и вариант задания.
3. Пошаговый порядок выполнения работы с комментариями и снимками экрана работы, а также изображение готовой схемы с подробным описанием.
4. Вывод.

Контрольные вопросы к защите работы:

1. Сущность объектно-ориентированного подхода.
2. Принципы объектно-ориентированного подхода.
3. Что такое проектирование ПО?
4. Что включает процесс проектирования ПО?
5. Охарактеризуйте понятие UML.
6. Каковы преимущества использования UML?
7. Перечислите виды диаграмм в UML.
8. Цель построения диаграммы деятельности?
9. Опишите основные элементы диаграммы деятельности. Какое графическое обозначение имеют элементы.
10. Объясните понятие деятельность в рамках языка UML?

Лабораторная работа №10

«Применение объектно-ориентированного подхода в анализе и проектировании ПО. Диаграмма последовательности».

Цель: построение диаграммы последовательности информационной системы в процессе проектирования ПО.

Порядок выполнения работы

1. Изучить теоретическую часть лабораторной работы.
2. Построить функциональную схему, отражающую состав и взаимодействие частей разрабатываемого программного обеспечения, в соответствии с заданием. Для построения использовать векторный графический редактор Microsoft Office Visio.
3. Проанализировать построенную схему.
4. Оформить отчет.

Варианты заданий аналогичны.

Теоретические сведения

Характеристика диаграмм последовательности. Рассмотренные диаграммы деятельности используются для спецификации динамики поведения программных систем, время в явном виде в них не присутствует. Однако временной аспект поведения может иметь существенное значение при моделировании синхронных процессов, описывающих взаимодействия объектов. Именно для этой цели в языке UML используются диаграммы последовательности.

Диаграмма последовательности системы — графическая модель, которая для определенного сценария варианта использования показывает динамику взаимодействия объектов во времени. Для построения диаграммы последовательности системы необходимо:

- идентифицировать каждое действующее лицо (объект) и изобразить для него линию жизни;
- из описания варианта использования определить множество системных событий и их последовательность;
- изобразить системные события в виде линий со стрелкой на конце между линиями жизни действующих лиц и системы, а также указать имена событий и списки передаваемых значений.

На диаграмме последовательности изображаются исключительно те объекты, которые непосредственно участвуют во взаимодействии и не показываются возможные статические ассоциации с другими объектами. Для диаграммы последовательности ключевым моментом является именно динамика взаимодействия объектов во времени. При этом диаграмма последовательности имеет как бы два измерения.

Одно измерение — слева направо в виде вертикальных линий, каждая из которых изображает линию жизни отдельного объекта, участвующего во взаимодействии. Графически каждый объект изображается прямоугольником и располагается в верхней части своей линии жизни. Внутри прямоугольника записываются имя объекта и имя

класса, разделенные двоеточием. При этом вся запись подчеркивается линией, что является признаком объекта, представляющим собой экземпляр класса (рис. 4.12).

Не исключается ситуация, когда имя объекта может отсутствовать на диаграмме последовательности. В этом случае указывается только имя класса, а сам объект считается анонимным. Крайним слева на диаграмме изображается объект, который является инициатором взаимодействия (объект 1 на рис. 4.12). Правее изображается другой объект, который непосредственно взаимодействует с первым. Таким образом, все объекты на диаграмме последовательности образуют порядок, определяемый степенью активности этих объектов при взаимодействии друг с другом.

Второе измерение диаграммы последовательности — вертикальная временная ось, направленная сверху вниз. Начальному моменту времени соответствует самая верхняя часть диаграммы.

При этом взаимодействия объектов реализуются посредством сообщений, которые посылаются одними объектами другим. Сообщения изображаются в виде горизонтальных стрелок с именем сообщения и также образуют порядок по времени своего возникновения.

Другими словами, сообщения, расположенные на диаграмме последовательности выше, инициируются раньше тех, которые расположены ниже. При этом масштаб на оси времени не указывается, поскольку диаграмма последовательности моделирует лишь временную упорядоченность взаимодействий типа «раньше-позже».

Линия жизни объекта служит для обозначения периода времени, в течение которого объект существует в системе и, следовательно, может потенциально участвовать во всех ее взаимодействиях.

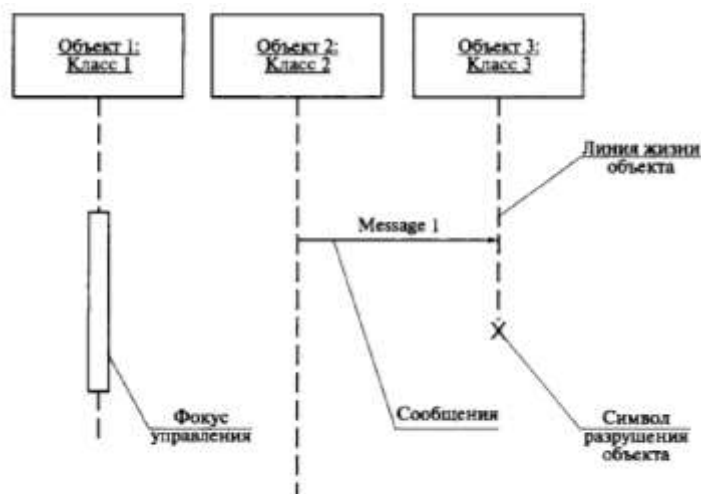


Рис. 4.12. Различные графические примитивы диаграммы последовательности

На диаграмме линия жизни изображается пунктирной вертикальной линией, ассоциированной с единственным объектом.

Если объект существует в системе постоянно, то и его линия жизни должна продолжаться по всей плоскости диаграммы последовательности.

Построение диаграммы последовательности целесообразно начинать с выделения из всей совокупности тех и только тех классов, объекты которых участвуют в моделируемом взаимодействии.

После этого все объекты наносятся на диаграмму с соблюдением некоторого порядка инициализации сообщений. Здесь необходимо установить, какие объекты будут существовать постоянно, а какие временно — только на период выполнения ими требуемых действий. Когда объекты определены, можно приступать к спецификации сообщений. При этом следует учитывать роли, которые сообщения играют в системе.

Пример построения диаграммы последовательности. Для того чтобы более точно учитывать временной аспект поведения системы при моделировании процессов, описывающих взаимодействие, можно построить диаграммы последовательности. В качестве примера построим диаграмму последовательности для реализации варианта использования «Продажа товара» в информационной системе «Склад оптовой торговли» (рис. 4.13).

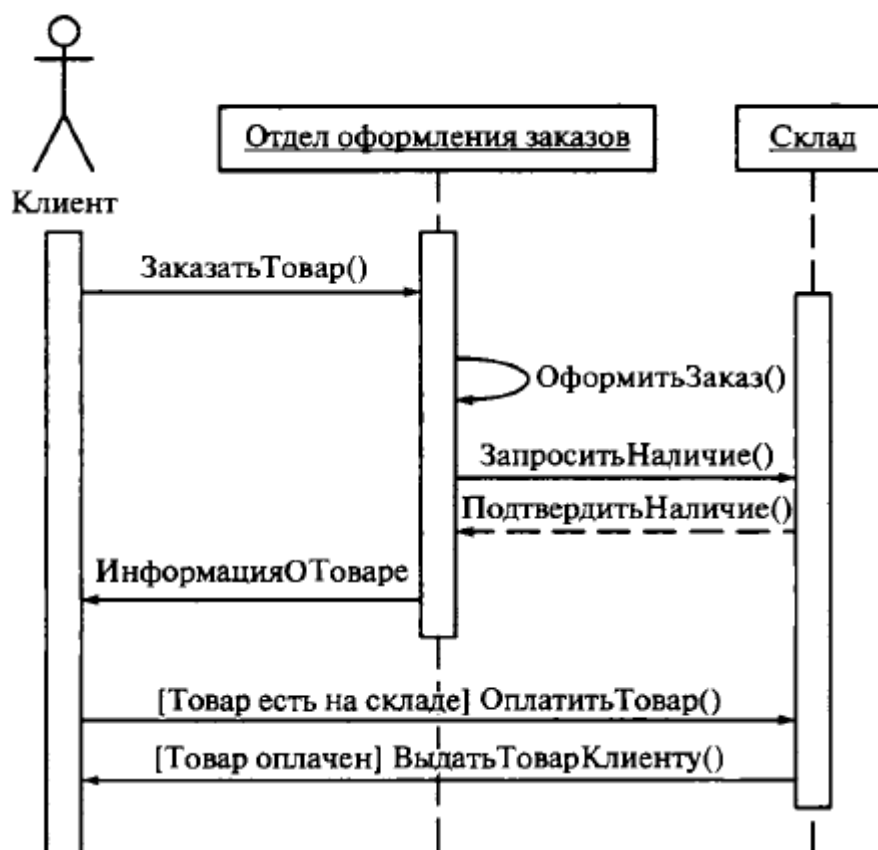


Рис. 4.13. Вариант диаграммы последовательности для моделирования операции продажи товара

Данная диаграмма содержит два объекта и одного актера.

Объекты не являются постоянно активными, что показано с помощью соответствующих фокусов управления. В качестве имен сообщений указаны имена операций, которые специфицированы у соответствующих классов. Предложения— условия некоторых сообщений записаны обычным текстом в квадратных скобках.

Эти условия отражают возможность ветвления процесса продажи и выполнения исключительного сценария соответствующего варианта использования, однако другие варианты использования на данной диаграмме не показаны.

Порядок оформления отчета по лабораторной работе

1. Введение.

Добавить теоретическую справку по теме.

2. Цель работы и вариант задания.
3. Пошаговый порядок выполнения работы с комментариями и снимками экрана работы, а также изображение готовой схемы с подробным описанием.
4. Вывод.

Контрольные вопросы к защите лабораторной работе:

1. Сущность объектно-ориентированного подхода.
2. Принципы объектно-ориентированного подхода.
3. Что такое проектирование ПО?
4. Что включает процесс проектирования ПО?
5. Охарактеризуйте понятие UML.
6. Каковы преимущества использования UML?
7. Перечислите виды диаграмм в UML.
8. Цель построения диаграммы последовательности?
9. Опишите основные элементы диаграммы последовательности. Какое графическое обозначение имеют элементы.
10. Что такое фокус управления?
11. Что такое линия жизни?
12. Какие типы связей в этой диаграмме вам известны?

Лабораторная работа №11

«Применение объектно-ориентированного подхода в анализе и проектировании ПО. Диаграмма классов».

Цель: построение диаграммы классов информационной системы в процессе проектирования ПО.

Порядок выполнения работы

1. Изучить теоретическую часть лабораторной работы.
2. Построить функциональную схему, отражающую состав и взаимодействие классов разрабатываемого программного обеспечения, в соответствии с заданием. Для построения использовать case-средство Rational Rose.
3. Проанализировать построенную схему.
4. Оформить отчет.

Варианты заданий аналогичны.

Теоретические сведения

Диаграммой UML, поясняющей внутреннее устройство системы, является диаграмма классов, описывающая создаваемую программную систему через ее компоненты (классы, объекты), отношения и взаимодействия между ними. В основном диаграммы классов в этих методах применяют на этапе проектирования, для того чтобы показать особенности построения конкретных классов.

UML предлагает использовать три уровня диаграмм классов в зависимости от степени их детализации:

- концептуальный уровень, на котором диаграммы классов, называемые в этом случае контекстными, демонстрируют связи между основными понятиями предметной области;
- уровень спецификаций, на котором диаграммы классов отображают интерфейсы классов предметной области, т. е. связи объектов этих классов;
- уровень реализации, на котором диаграммы классов непосредственно показывают поля и методы конкретных классов.

Это три разные модели, связь между которыми неоднозначна. Так, если концептуальная модель определяет некоторое понятие предметной области как класс, то это не означает, что для реализации этого понятия будет использован отдельный класс. Однако во всех трех моделях нас интересуют типы объектов (классы) и их статические отношения, что позволяет использовать единую нотацию.

Каждая из перечисленных моделей используется на конкретном этапе разработки программного обеспечения:

- концептуальная модель — на этапе анализа;
- диаграммы классов уровня спецификации — на этапе проектирования;
- диаграммы классов уровня реализации — на этапе реализации.

Концептуальные модели в соответствии с определением оперируют понятиями предметной области, атрибутами этих понятий и отношениями между ними. Класс при этом традиционно понимают как совокупность общих признаков некоторой группы объектов предметной области. В соответствии с этим определением на диаграмме классов

каждому классу соответствует группа объектов, общие признаки которых и фиксирует класс.

На диаграммах класс изображается в виде прямоугольника, внутри которого указано имя класса (рис. 4.14, а). При необходимости допускается указывать характеристики класса, например, атрибуты, используя специальные секции условного обозначения (рис. 4.14, б), а также операции класса (рис. 4.14, в).

В качестве атрибутов представляют некоторые, существенные с точки зрения решаемой задачи характеристики объектов, например, идентифицирующие значения (имя, номер). Для конкретного объекта атрибут всегда имеет конкретное значение (рис. 4.15).

Имя класса должно быть уникальным в пределах пакета, который описывается некоторой совокупностью диаграмм классов (возможно, одной диаграммой). Оно указывается в первой верхней секции прямоугольника. В дополнение к общему правилу наименования элементов языка UML имя класса записывается по центру секции имени полужирным шрифтом и должно начинаться с заглавной буквы. Рекомендуется в качестве имен классов использовать существительные, записанные по практическим соображениям без пробелов. Необходимо помнить, что именно имена классов образуют словарь предметной области при объектно-ориентированном анализе и проектировании.

В первой секции обозначения класса могут находиться ссылки на стандартные шаблоны или абстрактные классы, от которых образован данный класс и соответственно от которых он наследует свойства и методы. В этой секции может приводиться информация о разработчике данного класса и статус состояния разработки, а также могут записываться и другие общие свойства этого класса, имеющие отношение к другим классам диаграммы или стандартным элементам языка UML.

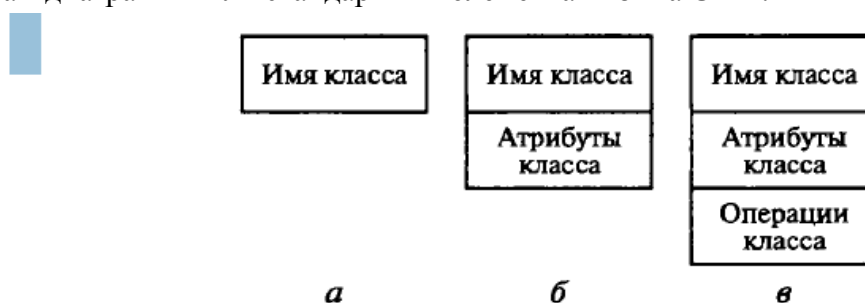


Рис. 4.14. Обозначение класса на концептуальной диаграмме классов:

***а* — без уточнения характеристик; *б* — с уточнением атрибутов; *в* — с указанием операций**

Примерами имен классов могут быть такие существительные, как «Сотрудник», «Компания», «Руководитель», «Клиент», «Продавец», «Менеджер», «Офис» и многие другие, имеющие непосредственное отношение к моделируемой предметной области и функциональному назначению проектируемой системы.

Класс может не иметь экземпляров или объектов. В этом случае он называется абстрактным классом, а для обозначения его имени используется наклонный шрифт (курсив). В языке UML принято общее соглашение о том, что любой текст, относящийся к абстрактному элементу, записывается курсивом.



Рис. 4.15. Пример графического изображения объектов на диаграммах языка UML

В некоторых случаях необходимо явно указать, к какому пакету относится тот или иной класс. Для этой цели используется специальный символ разделитель — двойное двоеточие. Синтаксис строки имени класса в этом случае будет следующий: <Имя_пакета>::<Имя_класса>.

Другими словами, перед именем класса должно быть явно указано имя пакета, к которому его следует отнести. Например, если определен пакет с именем «Банк», то класс «Счет» в этом банке может быть записан в виде: «Банк::Счет».

Во второй сверху секции прямоугольника класса записываются его атрибуты (attributes) или свойства. В языке UML принята определенная стандартизация записи атрибутов класса, которая подчиняется некоторым синтаксическим правилам. Каждому атрибуту класса соответствует отдельная строка текста, которая состоит из квантора видимости атрибута, имени атрибута, его кратности, типа значений атрибута и, возможно, его исходного значения:

<квантор видимости> <имя атрибута> [кратность] : <тип атрибута> = <исходное значение> {строка-свойство}

Квантор видимости принимает одно из трех возможных значений и отображается при помощи специальных символов:

- «+» — атрибут с областью видимости типа общедоступный (public). Атрибут доступен или виден из любого другого класса пакета, в котором определена диаграмма;
- «#» — атрибут с областью видимости типа защищенный (protected). Атрибут недоступен или невиден для всех классов за исключением подклассов данного класса;
- «-» — атрибут с областью видимости типа закрытый (private). Атрибут с этой областью видимости недоступен или невиден для всех классов без исключения.

Квантор видимости может быть опущен, в этом случае его отсутствие просто означает, что видимость атрибута не указывается.

Эта ситуация отличается от принятых по умолчанию соглашений в традиционных языках программирования, когда отсутствие квантора видимости трактуется как public или private. Однако вместо условных графических обозначений можно записывать соответствующее ключевое слово: public, protected, private.

Имя атрибута представляет собой строку текста, которая используется в качестве идентификатора соответствующего атрибута и потому должна быть уникальной в

пределах данного класса. Имя атрибута является единственным обязательным элементом синтаксического обозначения атрибута.

Кратность атрибута характеризует общее количество конкретных атрибутов данного типа, входящих в состав отдельного класса. В общем случае кратность записывается в форме строки текста в квадратных скобках после имени соответствующего атрибута.

В качестве примера рассмотрим следующие варианты задания кратности атрибутов: [0..1] означает, что кратность атрибута может принимать значение 0 или 1. При этом 0 означает отсутствие значения для данного атрибута;

[0..*] означает, что кратность атрибута может принимать любое положительное целое значение большее или равное 0. Эта кратность может быть записана короче в виде простого символа — [*];

[1..*] означает, что кратность атрибута может принимать любое положительное целое значение большее или равное 1;

[1..5] означает, что кратность атрибута может принимать любое значение из чисел: 1, 2, 3, 4, 5;

[1..3,5,7] означает, что кратность атрибута может принимать любое значение из чисел: 1, 2, 3, 5, 7;

[1..3,7..10] означает, что кратность атрибута может принимать любое значение из чисел: 1, 2, 3, 7, 8, 9, 10;

[1..3,7..*] означает, что кратность атрибута может принимать любое значение из чисел: 1, 2, 3, а также любое положительное целое значение большее или равное 7.

Если кратность атрибута не указана, то по умолчанию принимается ее значение равное 1..1, т. е. в точности 1.

Тип атрибута представляет собой выражение, семантика которого определяется языком спецификации соответствующей модели.

В нотации UML тип атрибута иногда определяется в зависимости от языка программирования, который предполагается использовать для реализации данной модели. В простейшем случае тип атрибута указывается строкой текста, имеющей осмысленное значение в пределах пакета или модели, к которым относится рассматриваемый класс.

При задании атрибутов могут быть использованы две дополнительные синтаксические конструкции — это подчеркивание строки атрибута и пояснительный текст в фигурных скобках.

Подчеркивание строки атрибута означает, что соответствующий атрибут может принимать подмножество значений из некоторой области значений атрибута, определяемой его типом. Эти или массив, которые в совокупности характеризуют каждый объект класса.

В третьей сверху секции прямоугольника записываются операции, или методы класса. Операция (operation) представляет собой некоторый сервис, предоставляющий каждый экземпляр класса по определенному требованию. Совокупность операций характеризует функциональный аспект поведения класса. Запись операций класса в языке UML также стандартизована и подчиняется определенным синтаксическим правилам. При этом каждой операции класса соответствует отдельная строка, которая состоит из квантора видимости операции, имени операции, выражения типа возвращаемого операцией значения и, возможно, строка—свойство данной операции:

<квантор видимости>имя операции> (список параметров) : <выражение типа возвращаемого значения>;» {строка-свойство}

Квантор видимости, как и в случае атрибутов класса, может принимать одно из трех возможных значений и соответственно отображается при помощи специального символа. Символ «+» обозначает операцию с областью видимости типа общедоступный (public). Символ «#» обозначает операцию с областью видимости типа защищенный (protected). И наконец, символ «-» используется для обозначения операции с областью видимости типа закрытый (private).

Квантор видимости для операции может быть опущен. В этом случае его отсутствие просто означает, что видимость операции не указывается. Вместо условных графических обозначений также можно записывать соответствующее ключевое слово: public, protected, private. Кроме внутреннего устройства или структуры классов, важную роль при разработке программной системы играют различные отношения между классами, которые также могут быть изображены на диаграмме классов:

- отношение ассоциации — наличие произвольной взаимосвязи между классами;
- отношение обобщения — отношение между более общим элементом и более частным элементом (родителем и потомком);
- отношение композиции (рис. 4.16, а) — частный случай отношения агрегации, когда части не могут выступать в отрыве от целого и с уничтожением целого уничтожаются и все его составные части;



Рис. 4.16. Условные обозначения специальных видов ассоциации:

а — композиция; б — агрегация

- отношение агрегации (рис. 4.16, б) — один из классов представляет собой некоторую сущность, которая включает в себя в качестве составных частей некоторые другие сущности.

В контексте языка UML существует специальный класс — интерфейс, у которого имеются только операции и отсутствуют атрибуты.

Интерфейсы на диаграмме служат для спецификации таких элементов модели, которые видимы извне, но их внутренняя структура остается скрытой от клиентов.

Процесс разработки диаграммы классов занимает центральное место при разработке проектов сложных систем. От умения правильно выбрать классы и установить между ними взаимосвязи зависит не только успех процессов проектирования, но и производительность программы.

Порядок оформления отчета по лабораторной работе

1. Введение.
Добавить теоретическую справку по теме.
2. Цель работы и вариант задания.
3. Пошаговый порядок выполнения работы с комментариями и снимками экрана работы, а также изображение готовой схемы с подробным описанием.
4. Вывод.

Контрольные вопросы к защите лабораторной работе:

1. Сущность объектно-ориентированного подхода.

2. Принципы объектно-ориентированного подхода.
3. Что такое проектирование ПО?
4. Что включает процесс проектирования ПО?
5. Охарактеризуйте понятие UML.
6. Каковы преимущества использования UML?
7. Перечислите виды диаграмм в UML.
8. Цель построения диаграммы классов?
9. Опишите основные элементы диаграммы классов. Какое графическое обозначение имеют элементы.
10. Что такое класс?
11. Что такое объект?
12. Какие типы связей в этой диаграмме вам известны?
13. Приведите примеры атрибутов ассоциаций. Что такое кратность ассоциации?
14. Какие сущности обычно содержат диаграммы классов?